VOL. 12

Next.js

Full-stack applications with the App Router, server components and server actions

Source	nextjs.org/learn
Generated	2026-10-10
Level	Intermediate
Estimated study time	30 hours
Language	English
Volume	Vol. 12 - Next.js

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 8
Chapter map 9
Chapter 1 - From React to Next.js 9
1.1 Summary 9
1.2 Concepts 10
What the framework adds 10
The request flow of the App Router 10
Server Components and Client Components 11
The two routers 11
What changed in Next.js 15 12
1.3 Examples 12
1.4 Common mistakes and best practices 15
1.5 Practice exercises 15
Solutions 15
1.6 Mini test 15
Answer key and explanations 16
Chapter 2 - Project setup and anatomy 16
2.1 Summary 16
2.2 Concepts 17
The folder tree 17
What each file is for 17
Special files in a route segment 17
`next.config.ts` 18
The scripts you use every day 18
2.3 Examples 18
2.4 Common mistakes and best practices 20
2.5 Practice exercises 20
Solutions 21
2.6 Mini test 21
Answer key and explanations 21
Chapter 3 - Styling with Tailwind, CSS Modules, fonts and images 21
3.1 Summary 21

3.2 Concepts . . . 22
Utility CSS with Tailwind . . . 22
CSS Modules . . . 22
The built-in font helpers . . . 22
`next/image` . . . 23
3.3 Examples . . . 23
3.4 Common mistakes and best practices . . . 26
3.5 Practice exercises . . . 26
Solutions . . . 27
3.6 Mini test . . . 27
Answer key and explanations . . . 27
Chapter 4 - Routing: layouts, pages and route segments . . . 27
4.1 Summary . . . 28
4.2 Concepts . . . 28
Folders to URLs . . . 28
How layouts nest . . . 28
Dynamic segments . . . 29
Route groups and private folders . . . 29
Route handlers . . . 29
4.3 Examples . . . 29
4.4 Common mistakes and best practices . . . 32
4.5 Practice exercises . . . 33
Solutions . . . 33
4.6 Mini test . . . 33
Answer key and explanations . . . 33
Chapter 5 - Navigating between pages with the Link component . . . 34
5.1 Summary . . . 34
5.2 Concepts . . . 34
What a link does . . . 34
Hooks and helpers for navigation . . . 35
Link options . . . 35
Rendering the whole payload, or the shell . . . 35
5.3 Examples . . . 35
5.4 Common mistakes and best practices . . . 37
5.5 Practice exercises . . . 38

Solutions . . . 38
5.6 Mini test . . . 38
Answer key and explanations . . . 38
Chapter 6 - Data access and fetching in server components . . . 39
6.1 Summary . . . 39
6.2 Concepts . . . 39
The data module pattern . . . 39
Caching options for `fetch` . . . 40
Sequential and parallel fetching . . . 40
Request deduplication with `cache` . . . 40
6.3 Examples . . . 40
6.4 Common mistakes and best practices . . . 42
6.5 Practice exercises . . . 43
Solutions . . . 43
6.6 Mini test . . . 43
Answer key and explanations . . . 44
Chapter 7 - Static, dynamic rendering and revalidation . . . 44
7.1 Summary . . . 44
7.2 Concepts . . . 45
The two rendering modes . . . 45
The dynamic APIs . . . 45
Segment configuration . . . 45
Invalidation . . . 46
7.3 Examples . . . 46
7.4 Common mistakes and best practices . . . 48
7.5 Practice exercises . . . 49
Solutions . . . 49
7.6 Mini test . . . 49
Answer key and explanations . . . 49
Chapter 8 - Streaming and loading states . . . 50
8.1 Summary . . . 50
8.2 Concepts . . . 50
What the browser receives . . . 50
The three levels of control . . . 50
Writing a fallback . . . 51

8.3 Examples . . . 51
8.4 Common mistakes and best practices . . . 53
8.5 Practice exercises . . . 53
Solutions . . . 53
8.6 Mini test . . . 54
Answer key and explanations . . . 54
Chapter 9 - Search and pagination . . . 54
9.1 Summary . . . 54
9.2 Concepts . . . 55
The shape of the query string . . . 55
Client or server reading . . . 55
Debouncing . . . 55
Pagination maths . . . 56
9.3 Examples . . . 56
9.4 Common mistakes and best practices . . . 60
9.5 Practice exercises . . . 60
Solutions . . . 60
9.6 Mini test . . . 60
Answer key and explanations . . . 61
Chapter 10 - Mutations with Server Actions . . . 61
10.1 Summary . . . 61
10.2 Concepts . . . 62
Where the directive goes . . . 62
The life of a mutation . . . 62
Errors and results . . . 62
Security . . . 63
10.3 Examples . . . 63
10.4 Common mistakes and best practices . . . 65
10.5 Practice exercises . . . 65
Solutions . . . 66
10.6 Mini test . . . 66
Answer key and explanations . . . 66
Chapter 11 - Forms and server-side validation . . . 66
11.1 Summary . . . 67

11.2 Concepts . . . 67
The state an action returns . . . 67
The two client helpers . . . 67
Version note . . . 68
Accessible error reporting . . . 68
11.3 Examples . . . 68
11.4 Common mistakes and best practices . . . 71
11.5 Practice exercises . . . 72
Solutions . . . 72
11.6 Mini test . . . 72
Answer key and explanations . . . 73
Chapter 12 - Errors, loading and not-found files . . . 73
12.1 Summary . . . 73
12.2 Concepts . . . 73
The boundary ladder . . . 73
The props of `error.tsx` . . . 74
`notFound()` and `not-found.tsx` . . . 74
12.3 Examples . . . 74
12.4 Common mistakes and best practices . . . 76
12.5 Practice exercises . . . 77
Solutions . . . 77
12.6 Mini test . . . 77
Answer key and explanations . . . 77
Chapter 13 - Authentication and protected routes . . . 78
13.1 Summary . . . 78
13.2 Concepts . . . 78
Where a check can live . . . 78
Reading a session on the server . . . 79
The middleware layer . . . 79
Forbidden and unauthorized responses . . . 79
13.3 Examples . . . 79
13.4 Common mistakes and best practices . . . 82
13.5 Practice exercises . . . 82
Solutions . . . 82
13.6 Mini test . . . 82

Answer key and explanations . . . 83
Chapter 14 - Metadata, SEO and deployment . . . 83
14.1 Summary . . . 83
14.2 Concepts . . . 84
Where the checks happen: the two exports . . . 84
Inheritance and templates . . . 84
File conventions . . . 84
Building and running . . . 84
14.3 Examples . . . 85
14.4 Common mistakes and best practices . . . 87
14.5 Practice exercises . . . 88
Solutions . . . 88
14.6 Mini test . . . 88
Answer key and explanations . . . 89
Cheat sheet . . . 89
15.1 Commands . . . 89
15.2 Files and conventions . . . 89
15.3 Components and navigation . . . 90
15.4 Rendering and caching . . . 90
15.5 Mutations and forms . . . 91
15.6 Metadata . . . 91
15.7 Version 15 at a glance . . . 92
15.8 Styling, fonts and images . . . 92
Final test . . . 92
16.1 Final test answers . . . 93
Sources . . . 94

How to use this volume

Prerequisites. HTML and CSS basics (Vol. 01 and Vol. 02), working modern JavaScript (Vol. 03) and the React fundamentals from Vol. 04: components, JSX, props, state, `useState` and `useEffect`, event handling and `async`/`await`. No Next.js experience is assumed.

Estimated study time. About 30 hours: roughly two hours per chapter plus the time you spend on the exercises. Chapters 1 to 7 carry the core model; chapters 8 to 14 cover rendering strategies, mutations, forms, authentication and shipping to production.

How the chapters work. Every chapter opens with a summary of its ideas, explains the concepts with tables and diagrams, shows numbered examples with the output they produce, lists the common mistakes next to the best practice for each, and closes with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. Blocks tagged `tsx` are components you paste into the `app/` folder of a project, `ts` blocks are plain modules such as `app/lib/data.ts`, `json` and `bash` blocks are configuration and terminal commands, and `text` blocks show what you see in the terminal or the browser. Creating a project needs a network connection, because `create-next-app` downloads the dependencies and `next/font/google` downloads fonts at build time.

shell

```shell
npx create-next-app@latest learn-next --ts --tailwind --eslint --app --import-alias "@/*"
cd learn-next
npm run dev
```

output

```text
? Would you like to use TypeScript? ... No / Yes
? Would you like to use ESLint? ... No / Yes
? Would you like to use Tailwind CSS? ... No / Yes
? Would you like your code inside a `src/` directory? ... No / Yes
? Would you like to use App Router? (recommended) ... No / Yes
? Would you like to use Turbopack for `next dev`? ... No / Yes
? Would you like to customize the import alias (`@/*` by default)? ... No / Yes

Creating a new Next.js app in C:\projects\learn-next.

Downloading files. This might take a moment.

Initialized a git repository.

Success! Created learn-next at C:\projects\learn-next
```

The wording of the prompts changes slightly between minor versions; the command line flags above answer every one of them without interaction.

About the source. The syllabus follows the official learning path at nextjs.org/learn, whose main track builds a dashboard application. The theme of the examples, an invoicing dashboard, is reused here, but every sentence and every line of code was written for this volume. Deployment is described in a platform-neutral way, with notes where a hosting account is assumed.

Chapter map

#	Chapter	You will learn
1	From React to Next.js	what the framework adds, the two routers, server and client components, the version 15 changes
2	Project setup and anatomy	`create-next-app`, the folder tree, `next.config.ts`, dev and build commands
3	Styling with Tailwind, CSS Modules, fonts and images	utility CSS, CSS Modules, `next/font`, `next/image`
4	Routing: layouts, pages and route segments	nested routes, layouts, dynamic segments, route groups, route handlers
5	Navigating between pages with `<Link>`	client-side navigation, active links, programmatic navigation
6	Data access and fetching in server components	a data module, `async` components, `fetch`, `Promise.all`, request deduplication
7	Static, dynamic rendering and revalidation	prerendering, dynamic APIs, `revalidate`, `revalidatePath` and `revalidateTag`
8	Streaming and loading states	`loading.tsx`, `Suspense`, skeletons, partial rendering
9	Search and pagination	`searchParams`, the URL as state, debouncing, `LIMIT` and `OFFSET`
10	Mutations with Server Actions	`"use server"`, form actions, `revalidatePath`, `redirect`
11	Forms and server-side validation	`useActionState`, `useFormStatus`, Zod, accessible errors
12	Errors, loading and not-found files	`error.tsx`, `global-error.tsx`, `not-found.tsx`, `notFound()`
13	Authentication and protected routes	sessions, server-side checks, middleware, protecting mutations
14	Metadata, SEO and deployment	the metadata API, `generateMetadata`, `next build`, self-hosting
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - From React to Next.js

1.1 Summary

- React describes interfaces with components, but leaves routing, rendering, data access and deployment to other tools.
- Next.js is a framework built on React that adds a file-system router, a rendering pipeline, caching rules, data helpers and production optimizations.
- The App Router, kept in the `app/` folder, is the current router and the subject of this volume; the older Pages Router lives in `pages/` and is summarised in Chapter 14.
- Every component in the App Router is a Server Component until a file starts with the `"use client"` directive, which turns that file and everything it imports into client code.
- Server Components run only on the server, can read the database and secrets directly, and never ship their own JavaScript to the browser.
- Client Components are the React you already know: state, effects, event handlers and browser APIs, executed in the browser.
- The two kinds of component compose into one tree, and the `"use client"` line is the boundary where the JavaScript bundle is cut.
- The React Foundations track that precedes this course covers components, JSX, props, state, events, ES modules and `async`/`await`, all of which this volume assumes.
- This volume describes Next.js 15, which runs on React 19; the places where the API differs from version 14 are flagged so older tutorials can be adapted.

1.2 Concepts

What the framework adds

React answers the question "how do I describe the screen". A production application also needs routing, rendering, data, images and a server, so Next.js supplies those pieces through conventions rather than extra libraries:

Concern	React alone	Next.js App Router
Routing	an external library	folders and files under `app/`
Rendering	browser only	server, static and streaming rendering
Layouts	composition by hand	nested `layout.tsx` files
Data	your own `fetch` calls	`async` components, `fetch`, caching rules
Metadata	hand-written tags	the metadata API
Images	manual sizing	`next/image`
Fonts	manual hosting	`next/font`
API endpoints	a server you write	`route.ts` handlers
Bundling	your own toolchain	a build with code splitting

The request flow of the App Router

The router reads the URL, finds the matching tree of files, renders it on the server, and streams the result to the browser:

output

```
browser asks for /dashboard/invoices
                |
                v
app/                          layouts offer a shell, pages offer content
  layout.tsx        -------->  <html><body> ...
  dashboard/layout.tsx  ---->  sidebar + header
    invoices/page.tsx  ---->  the page itself
                |
                v
the server renders the tree; "use client" marks the parts that
also ship as JavaScript to hydrate in the browser
                |
                v
HTML streams to the browser, then hydration makes the
interactive parts respond to clicks
```

Two consequences fall out of that picture. First, a page can spend most of its life waiting for the database while the surrounding shell has already reached the user; Chapter 8 turns that into a loading experience. Second, whatever is rendered on the server can be rendered again for the next request, which is why the caching rules in Chapter 7 deserve attention.

Server Components and Client Components

	Server Component	Client Component
Directive	none, it is the default	`"use client"` on the first line of the file
Runs	on the server only	in the browser (and on the server while prerendering)
State, effects, events	not available	available
Database and secrets	direct access	never, use a server action or a route handler
JavaScript sent to the browser	none	yes
Can render the other kind	only as children passed through props	yes, any component

The working rule is to stay on the server unless a component needs interactivity, a hook, or a browser API.

The two routers

	App Router (`app/`)	Pages Router (`pages/`)
Status	current and recommended	maintained, but legacy
Default component kind	server component	client component
Data fetching	`async` components	`getServerSideProps`, `getStaticProps`

	App Router (`app/`)	Pages Router (`pages/`)
Layouts	nested `layout.tsx`	one `_app` file plus per-page wrappers
Streaming	built in	not available
New features	yes	frozen

Both folders can live in the same project, and Next.js prefers `app/` when a path matches in both. New work belongs in `app/`; the Pages Router appears in Chapter 14 only because so much existing code and so many tutorials still use it.

What changed in Next.js 15

This volume targets Next.js 15 with React 19. These are the differences you meet in the first hours of using the framework:

Area	Version 14	Version 15
`params` in pages, layouts, route handlers	a plain object	a `Promise` that you `await`
`searchParams` in pages	a plain object	a `Promise` that you `await`
`cookies()`, `headers()`, `draftMode()`	synchronous	asynchronous functions
`fetch` caching	GET requests cached by default	not cached by default
GET route handlers	cached by default	not cached by default
Form state hook	`useFormState` from `react-dom`	`useActionState` from `react`
Client router cache	dynamic pages stale after 30 s	no stale time by default

When a snippet from an older tutorial does not behave as documented, this table is the first place to look.

1.3 Examples

Example 1: a page written as a server component.

TSX

```tsx
// app/invoices/page.tsx
import { getInvoices } from '@/app/lib/data';

export default async function InvoicesPage() {
  const invoices = await getInvoices();
  return (
    <main>
      <h1>Invoices</h1>
      <table>
        <tbody>
          {invoices.map((invoice) => (
            <tr key={invoice.id}>
              <td>{invoice.name}</td>
              <td>{invoice.amount}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </main>
  );
}
```

output

```
The browser receives finished HTML: one heading and one table row per invoice.
No invoice-fetching code is present in the JavaScript bundle.
```

The component is `async`, so it can wait for the database directly. The helper it imports may hold a database password precisely because the file is a server component.

Example 2: a client island inside that tree.

TSX

```tsx
// app/components/search-field.tsx
'use client';

import { useState } from 'react';

export default function SearchField() {
  const [text, setText] = useState('');
  return (
    <input
      value={text}
      onChange={(event) => setText(event.target.value)}
      placeholder="Search invoices"
    />
  );
}
```

output

```
The input keeps its own state and rerenders on every keystroke. Its JavaScript
is downloaded by the browser, because the file starts with "use client".
```

Example 3: the same route in both routers.

TSX

```tsx
// app/about/page.tsx
export default function AboutPage() {
  return <h1>About</h1>;
}
```

TSX

```tsx
// pages/about.jsx
export default function AboutPage() {
  return <h1>About</h1>;
}
```

output

```
Both files answer for /about. The App Router component is rendered on the
server; the Pages Router component runs in the browser after hydration, exactly
like a plain React component.
```

Example 4: a server page that hands plain data to a client child.

TSX

```tsx
// app/invoices/page.tsx
import { getInvoices } from '@/app/lib/data';
import InvoiceFilter from './invoice-filter';

export default async function InvoicesPage() {
  const invoices = await getInvoices();
  // Only serialisable data crosses the boundary.
  return <InvoiceFilter invoices={invoices} />;
}
```

TSX

```tsx
// app/invoices/invoice-filter.tsx
'use client';

import { useState } from 'react';

export default function InvoiceFilter({ invoices }) {
  const [query, setQuery] = useState('');
  const shown = invoices.filter((i) => i.name.includes(query));
  return (
    <>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <ul>
        {shown.map((i) => (
          <li key={i.id}>{i.name}</li>
        ))}
      </ul>
    </>
  );
}
```

output

```
The page's data is fetched once on the server and passed down as props; only
the filtering logic runs in the browser.
```

What crosses the boundary must be serialisable: strings, numbers, booleans, arrays and plain objects. Functions, class instances and database connections are not props.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Fetching in `useEffect` when the server could fetch	exposes the service and bloats the bundle	fetch in an `async` server component
Importing a server-only module from a client component	the secret ends up in the bundle	move it behind a server action or a route handler
Passing a function as a prop from server to client	the payload cannot be serialised	pass plain data, keep behaviour on the client
Copying version 14 code unchanged	`params` and `searchParams` have become promises	read them with `await`
Assuming a page is static or dynamic	wrong behaviour at deploy time and on the first visit	inspect the build output, or force the mode
Reaching for a client component by habit	the browser does work the server could do	start server-side, add `"use client"` only when needed

1.5 Practice exercises

1. Which two folders can hold routes in a Next.js project, and which one should new work use?
2. A page fetches data inside `useEffect` after mount. Which kind of component should it become, and what marks the change?
3. Name three things a Server Component may do that a Client Component may not.
4. Which values may a server component pass as props to a client component?
5. In Next.js 15, how do you read the id of a route segment from `app/invoices/[id]/page.tsx`?

Solutions

1. `app/` for the App Router and `pages/` for the Pages Router; new work uses `app/`, while `pages/` exists for legacy code.
2. It should become a server component, because the data does not depend on the browser. Nothing marks the change: dropping `"use client"` and removing the effect is enough.
3. Read the database or the filesystem, read secrets and environment variables, and import server-only packages.
4. Serialisable values only: strings, numbers, booleans, `null`, arrays, plain objects and dates.
5. The component receives a `params` prop whose type is a promise, so `const { id } = await params;` inside the async component.

1.6 Mini test

1. Which first line turns a component into a Client Component? A) `"use server"` B) `"use client"` C) `export const dynamic` D) `"use dom"`

2. True or false: a Server Component may call `useState`.
3. Which folder holds the files of the App Router? A) `src/` B) `components/` C) `app/` D) `pages/`
4. In Next.js 15, what is `params` inside a page component? A) a plain object B) a promise C) a function D) a string
5. Which of these may only a Server Component do? A) handle a click B) use `useEffect` C) read a database secret D) keep local state
6. True or false: if a client component imports a module, that module also becomes client code.
7. What kind of component is a file with no directive at the top, in the `app/` folder?

Answer key and explanations

1. B - the `"use client"` directive opens the client bundle boundary for that file and its imports.
2. False - hooks run in the browser, and a server component never ships JavaScript.
3. C - the App Router reads every route, layout and handler from `app/`.
4. B - in version 15 `params` is a promise, so it is awaited inside the component.
5. C - secrets must stay on the server; the other three are browser features.
6. True - the directive is a boundary, so everything imported below it is bundled for the browser.
7. A Server Component - server rendering is the default in the App Router.

Chapter 2 - Project setup and anatomy

2.1 Summary

- `npx create-next-app` scaffolds a complete project: router choice, TypeScript, ESLint, Tailwind, a dev server and a git repository.
- The `app/` folder is the router, so every folder inside it is a URL segment and special files decide what that segment renders.
- A root layout that renders `<html>` and `<body>` is mandatory, and the home route needs a `page.tsx`.
- `next.config.ts` holds build configuration such as image domains, redirects, headers and the bundle output mode.
- `npm run dev` serves the app on port 3000 with fast refresh; `npm run build` writes the production output into `.next`.
- The `@/*` import alias points at the project root, or at `src/` when that layout is chosen at scaffolding time.
- Files placed in `public/` are served unchanged from the root of the site, so `public/logo.svg` answers for `/logo.svg`.
- The scripts in `package.json` are the only commands you need from day to day.

2.2 Concepts

The folder tree

A fresh App Router project looks like this:

output

```
learn-next/
  app/
    favicon.ico
    globals.css
    layout.tsx
    page.tsx
  public/
    next.svg
    vercel.svg
  eslint.config.mjs
  next.config.ts
  package.json
  postcss.config.mjs
  tsconfig.json
```

Nothing in that tree is a mystery: the router is `app/`, the static files are `public/`, and the rest is tooling.

What each file is for

File	Job
`app/layout.tsx`	the required root layout; renders `<html>` and `<body>`
`app/page.tsx`	the user interface of the route the folder stands for
`app/globals.css`	global styles, imported by the root layout
`app/favicon.ico`	served as the site icon
`public/`	static files, one URL path each
`next.config.ts`	build configuration for the framework itself
`postcss.config.mjs`	usually Tailwind's plugin entry point
`tsconfig.json`	TypeScript options and the `@/*` path alias
`eslint.config.mjs`	lint rules, including the Next.js plugin

Special files in a route segment

A folder under `app/` renders according to which special files it contains:

File	Meaning
`layout.tsx`	a shell shared by every route below it
`page.tsx`	the content of the route itself
`loading.tsx`	the fallback shown while the segment streams in

File	Meaning
`error.tsx`	the client error boundary for the segment
`not-found.tsx`	the interface shown when `notFound()` is called
`route.ts`	an HTTP endpoint instead of a page
`template.tsx`	like a layout, but remounted on navigation

`next.config.ts`

The configuration file is ordinary TypeScript and is typed, so editors can complete the options. Anything that concerns the build rather than your application code goes here:

TypeScript

```ts
// next.config.ts
import type { NextConfig } from 'next';

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: 'images.example.com' },
    ],
  },
  async redirects() {
    return [{ source: '/home', destination: '/', permanent: true }];
  },
};

export default nextConfig;
```

Older projects use `next.config.mjs`; both forms are read by the same configuration loader.

The scripts you use every day

Script	Effect
`npm run dev`	dev server on port 3000 with fast refresh
`npm run build`	production build into `.next`
`npm run start`	serve the production build locally
`npm run lint`	run ESLint over the project

2.3 Examples

Example 1: create the project with the flags that answer every prompt.

shell

```
npx create-next-app@latest learn-next --ts --tailwind --eslint --app --import-alias "@/*"
```

output

```
Success! Created learn-next at C:\projects\learn-next
```

Example 2: start the dev server.

shell

```
cd learn-next
npm run dev
```

output

```
   ▲ Next.js 15.x
   - Local:        http://localhost:3000
   - Environments: .env

  ✓ Starting...
  ✓ Ready in 1.2s
```

Open http://localhost:3000 and the default page appears; editing any file refreshes the browser without losing component state.

Example 3: the root layout the tool created.

TSX

```tsx
// app/layout.tsx
import type { Metadata } from 'next';
import './globals.css';

export const metadata: Metadata = {
  title: 'Learn Next',
  description: 'An invoicing dashboard built while learning Next.js',
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

output

```
Every page of the application is rendered inside this shell, so the language
attribute, the title template and any provider live here once.
```

Example 4: the page that answers for `/`.

TSX

```tsx
// app/page.tsx
export default function HomePage() {
  return <h1>Welcome to the dashboard</h1>;
}
```

output

```
The browser shows a level-one heading reading "Welcome to the dashboard",
and the document title reads "Learn Next" from the layout metadata.
```

Example 5: the production build.

shell

```
npm run build
```

output

```
   ▲ Next.js 15.x

   Creating an optimized production build ...
 ✓ Compiled successfully
 ✓ Linting and checking validity of types
 ✓ Collecting page data
 ✓ Generating static pages (5/5)
 ✓ Collecting build traces
 ✓ Finalizing page optimization

Route (app)                    Size  First Load JS
┌ ○ /                        132 B         102 kB
└ ○ /about                   132 B         102 kB
○  (Static)  prerendered as static content
```

The circle and the lambda marker next to each route report how that route is rendered: a static page is prerendered once, a dynamic page is rendered per request.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Deleting the root layout	the app has nothing to render into	keep exactly one root layout in `app/`
Putting a `page.tsx` inside a route group by mistake	the route is not reachable	read the build output's route table after adding a folder
Editing files in `.next/`	they are overwritten on every build	treat `.next` as a build artefact
Serving assets from `app/`	only special files are routed there	move pure assets into `public/`
Hard-coding the port	it may already be in use	use `npm run dev -- -p 3001`
Committing `.env` files	secrets leak into the repository	keep them out of version control

2.5 Practice exercises

1. Which file must exist under `app/` for the application to build, and why?
2. Where does a file `public/avatar.png` become reachable, and at which URL?
3. Which command serves the production build, and which folder does the build write to?
4. You need to allow images from `https://cdn.example.com`. In which file do you add that domain?
5. What is the difference between `loading.tsx` and `page.tsx` in the same folder?

Solutions

1. `app/layout.tsx`: it renders the `<html>` and `<body>` elements that every other route is placed inside.
2. It becomes reachable at `http://localhost:3000/avatar.png`, because everything under `public/` is copied to the root of the site.
3. `npm run start` serves the build that `npm run build` wrote into `.next/`.
4. `next.config.ts`, under the `images.remotePatterns` option.
5. `page.tsx` is the content of the route; `loading.tsx` is the fallback shown while that content is still streaming in.

2.6 Mini test

1. Which command creates a new Next.js project from the terminal? A) `npm create next-app` B) `npx create-next-app` C) `npm install next` D) `npx next new`
2. Which file renders the `<html>` element of the application? A) `app/page.tsx` B) `app/layout.tsx` C) `index.html` D) `app/template.tsx`
3. True or false: static files should be placed in `app/assets`.
4. What does the `@/*` alias usually stand for? A) the `node_modules` folder B) the `public` folder C) the project root, or `src/` when it is used D) the `.next` folder
5. Which folder does the framework own and overwrite on every build? A) `public/` B) `app/` C) `.next/` D) `components/`
6. What is the default port of the dev server? A) 8080 B) 5173 C) 4200 D) 3000
7. Which configuration option allows remote images in `next/image`? A) `images.remotePatterns` B) `assets.domains` C) `static.allowed` D) `images.hosts`

Answer key and explanations

1. B - the scaffolding tool is `create-next-app`, normally run through `npx`.
2. B - the root layout is the only place where the document shell is defined.
3. False - static files belong in `public/`; `app/` is reserved for routing files.
4. C - the alias is configured in `tsconfig.json` by the scaffolding tool.
5. C - `.next/` is a build artefact and should never be edited.
6. D - port 3000, unless something else already uses it.
7. A - `images.remotePatterns` lists the protocols, hosts and path patterns that may be optimised.

Chapter 3 - Styling with Tailwind, CSS Modules, fonts and images

3.1 Summary

- A scaffolded project ships with Tailwind CSS: utility classes in the markup, one global stylesheet and a PostCSS plugin in the build.
- Tailwind version 4 configures itself from CSS, so the global file imports the framework and declares design tokens with `@theme`.
- A CSS Module scopes every class name it defines, so `styles.card` cannot collide with a `card` class in another file.
- `next/font` moves font hosting into the build: `next/font/google` downloads the files while building, `next/font/local` serves files kept in the repository.
- Both font helpers return a `className`, and both declare size-adjusted fallbacks so text does not shift when the real font arrives.
- `next/image` resizes, optimises and lazy-loads images, and it demands either intrinsic dimensions or a `fill` layout.
- A locally imported image provides its own width, height and blur placeholder automatically.
- Remote images must be listed under `images.remotePatterns` in `next.config.ts` before they can be optimised.
- The `priority` prop opts an image out of lazy loading, which suits the largest picture above the fold.

3.2 Concepts

Utility CSS with Tailwind

Utility classes describe the design in the markup. A button is styled where it is used, and removing the element removes its styles with it:

HTML

```
<button class="rounded-md bg-blue-600 px-4 py-2 text-white hover:bg-blue-500">
  Create invoice
</button>
```

The trade-off is a long `className`. When the same combination repeats, extract a component rather than a class, or use a helper that joins names.

CSS Modules

A CSS Module is a normal stylesheet whose class names are rewritten at build time. Importing it gives you an object of the local names:

Rule	Effect
`.card { ... }` in `styles.module.css`	available as `styles.card`
`:global(.shell) { ... }`	the name is kept as written and applies everywhere
`composes: card;`	reuse another rule from the same or another module
one file per component	keeps the cascade small and predictable

The built-in font helpers

	`next/font/google`	`next/font/local`
Source	the Google Fonts library	font files in your repository
Download	at build time, self-hosted afterwards	never
Runtime request to a third party	none	none
Typical use	`Inter({ subsets: ['latin'] })`	`localFont({ src: './font.woff2' })`
Variable fonts	supported with `variable`	supported with `variable`

Both return an object with a `className`, and both accept `weight`, `style`, `display` and `preload`.

`next/image`

`next/image` renders a responsive `img` with the right format and size for each device. Its contract is small:

Prop	Purpose
`src`	a static import, or a path string for remote images
`alt`	required; describe the image or leave it empty for decoration
`width`, `height`	intrinsic dimensions, known automatically for imported files
`fill`	stretch to the nearest positioned parent instead
`sizes`	tells the browser how wide the image will render
`priority`	skip lazy loading for above-the-fold images
`placeholder="blur"`	show a blur while loading; free for imported images

3.3 Examples

Example 1: the global stylesheet of a Tailwind v4 project.

CSS

```css
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #0969da;
  --font-sans: system-ui, sans-serif;
}

body {
  font-family: var(--font-sans);
}
```

output

```
The PostCSS plugin listed in postcss.config.mjs turns this file into the
utilities your markup uses. Design tokens declared in @theme become utilities
such as bg-brand.
```

A project on Tailwind v3 instead writes three directives, `@tailwind base; @tailwind components; @tailwind utilities;`, and keeps its theme in `tailwind.config.ts`. The two versions are not interchangeable, so check the installed version before copying a configuration.

Example 2: a styled card with utility classes.

TSX

```tsx
// app/ui/invoice-card.tsx
export default function InvoiceCard({ name, amount }: { name: string; amount: number }) {
  return (
    <div className="rounded-lg border border-gray-200 p-4 shadow-sm">
      <p className="font-semibold text-gray-900">{name}</p>
      <p className="text-sm text-gray-600">{amount} EUR</p>
    </div>
  );
}
```

output

```
A bordered, rounded card with a bold customer name and a muted amount line.
```

Example 3: the same card as a CSS Module.

CSS

```css
/* app/ui/invoice-card.module.css */
.card {
  border: 1px solid #d8dee4;
  border-radius: 8px;
  padding: 16px;
}

.name {
  font-weight: 600;
}

.amount {
  color: #57606a;
  font-size: 14px;
}
```

TSX

```tsx
// app/ui/invoice-card-module.tsx
import styles from './invoice-card.module.css';

export default function InvoiceCard({ name, amount }: { name: string; amount: number }) {
  return (
    <div className={styles.card}>
      <p className={styles.name}>{name}</p>
      <p className={styles.amount}>{amount} EUR</p>
    </div>
  );
}
```

output

```
The rendered markup contains rewritten class names, so the same file can be
imported by two components without any collision:

<div class="invoice-card-module__card__hK3f1">
  <p class="invoice-card-module__name__aQ9z2">Acme</p>
  <p class="invoice-card-module__amount__Lm4t0">250 EUR</p>
</div>
```

Example 4: a Google font in the root layout.

TSX

```tsx
// app/layout.tsx
import type { Metadata } from 'next';
import { Inter } from 'next/font/google';
import './globals.css';

const inter = Inter({ subsets: ['latin'], display: 'swap' });

export const metadata: Metadata = { title: 'Learn Next' };

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={inter.className}>
      <body>{children}</body>
    </html>
  );
}
```

output

```
During `npm run build` the font files are downloaded once and stored with the
build output. The browser then loads them from your own domain, with no request
to a third party and no invisible text while the font loads.
```

Example 5: a font file kept in the repository.

TypeScript

```ts
// app/ui/fonts.ts
import localFont from 'next/font/local';

export const brand = localFont({
  src: './brand-regular.woff2',
  weight: '400',
  style: 'normal',
  display: 'swap',
  variable: '--font-brand',
});
```

output

```
The returned className (or variable, when `variable` is set) is applied to the
element that should use the font. The file is copied into the build and served
from your own domain.
```

Example 6: images with `next/image`.

TSX

```tsx
// app/dashboard/page.tsx
import Image from 'next/image';
import hero from '@/public/hero.png';

export default function DashboardPage() {
  return (
    <>
      <Image src={hero} alt="Revenue chart for the current month" priority />
      <div className="relative h-48 w-full">
        <Image
          src="https://images.example.com/team.jpg"
          alt="The team at the office"
          fill
          sizes="(max-width: 768px) 100vw, 50vw"
        />
      </div>
    </>
  );
}
```

output

```
The local image is resized while building, and its width and height are read
from the file itself. The remote image fills its parent box, and the browser
picks the smallest variant that matches the declared sizes.
```

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Using a plain `img` tag	no resizing, no lazy loading, layout shift	use `next/image`
Omitting `alt`	screen readers announce a file name	write a description, or an empty string for decoration
`fill` without a sized parent	the image collapses or overflows	give the parent a position and a size
Remote host not declared in the config	the build fails on the image	add it to `images.remotePatterns`
Mixing Tailwind v3 and v4 configuration	classes silently do nothing	follow the installed version's setup
Loading webfonts from a third party	extra requests and privacy noise	use `next/font`

3.5 Practice exercises

1. Which two styling approaches does a new project support, and how do their class names stay unique?
2. Which two font helpers exist, and what is the difference between them?
3. What are the two ways to size an `next/image`, and which one needs a positioned parent?

4. Why does `next/image` require an `alt` attribute?
5. You must display a hero image above the fold. Which prop should you add, and why?

Solutions

1. Tailwind, where class names are global utilities used in the markup, and CSS Modules, where class names are rewritten per file. Modules keep uniqueness by construction; Tailwind relies on the generated utility names.
2. `next/font/google`, which downloads the font at build time from the Google library, and `next/font/local`, which serves a file stored in the repository.
3. Intrinsic `width` and `height`, or `fill`. The `fill` variant needs a parent with a position and an explicit size.
4. Because an image without alternative text is announced by its file name, or not at all; decoration should use an empty string instead.
5. `priority`, so the image is loaded eagerly and does not wait for lazy loading to kick in.

3.6 Mini test

1. Which import brings in the font loader for the Google Fonts library? A) `next/font/google` B) `next/google/fonts` C) `next/fonts` D) `next/image`
2. True or false: CSS Module class names are rewritten per file.
3. What does `next/image` need when `fill` is used? A) nothing else B) a parent with a position and a size C) a `sizes` prop D) a `width` prop
4. Which option lists the hosts allowed for remote images? A) `images.domains` in `package.json` B) `images.remotePatterns` in `next.config.ts` C) `allowedHosts` in `tsconfig.json` D) `public.hosts` in `next.config.ts`
5. When does `next/font/google` download the font files? A) on every page view B) at build time C) when `priority` is set D) never
6. Which directive starts a Tailwind v4 stylesheet? A) `@tailwind base` B) `@import "tailwindcss"` C) `@apply tailwind` D) `@use tailwind`

Answer key and explanations

1. A - `next/font/google` is the documented entry point for Google fonts.
2. True - the build rewrites the names, which is why the imports never collide.
3. B - `fill` positions the image absolutely inside its nearest positioned ancestor, so that ancestor must have a size.
4. B - the allow list lives in the framework configuration.
5. B - the files are fetched during the build and self-hosted afterwards.
6. B - version 4 is configured from CSS with a single import.

Chapter 4 - Routing: layouts, pages and route segments

4.1 Summary

- Every folder under `app/` is a URL segment, and every `page.tsx` inside it makes that segment publicly reachable.
- A folder without a `page.tsx` is not a route: it can still hold a layout, but the URL itself answers with a not-found page.
- Layouts nest from the root down, and a layout keeps its state and its scroll position across navigations inside its own segment.
- A layout receives its nested routes as `children`, which is how shared sidebars, headers and providers are placed once.
- Dynamic segments are folders named `[id]`, and in version 15 their values arrive as a `params` promise that the component awaits.
- `generateStaticParams` prerenders a known list of dynamic segments at build time, which is how a blog or a detail page becomes static.
- Route groups, written `(shop)`, organise files in the editor without adding anything to the URL.
- A private folder, written `_components`, is never routable and is the place for helpers used by a segment.
- A `route.ts` file exports HTTP verbs instead of a page, so the folder stops rendering a user interface there.

4.2 Concepts

Folders to URLs

File	URL
`app/page.tsx`	`/`
`app/about/page.tsx`	`/about`
`app/blog/page.tsx`	`/blog`
`app/blog/[slug]/page.tsx`	`/blog/hello-world`
`app/shop/[...parts]/page.tsx`	`/shop/a`, `/shop/a/b` and deeper
`app/(marketing)/pricing/page.tsx`	`/pricing`
`app/api/invoices/route.ts`	`/api/invoices`

The rule to remember is that only `page.tsx` and `route.ts` create a URL. Everything else in the folder is support.

How layouts nest

output

```
app/layout.tsx                 <html><body>
  app/dashboard/layout.tsx     sidebar + header
    app/dashboard/invoices/layout.tsx
      app/dashboard/invoices/page.tsx      the page itself
```

Each layout wraps the next one through `children`, so the dashboard shell is rendered once and reused while the user moves between invoicing pages.

Dynamic segments

Folder	Meaning
`[id]`	one required value, `params.id`
`[...slug]`	one or more values, `params.slug` is an array
`[[...slug]]`	optional catch-all, also matches the parent URL

Since version 15 the props are promises, which is why the examples below always await them.

Route groups and private folders

Name	Effect
`(marketing)`	no URL segment; groups files for a section or a shared layout
`(shop)`	the same, for another section
`_lib`	private, excluded from routing entirely
`@modal`	a named slot, rendered by the parent layout

Route groups solve the awkward case where two sections need different layouts but share the same URL prefix.

Route handlers

`route.ts` exports at most one function per HTTP verb. In version 15 the `params` of a dynamic handler is a promise as well, and a GET handler is not cached unless you ask for it.

4.3 Examples

Example 1: a nested route tree.

output

```
app/
  layout.tsx                /
  page.tsx                  /
  dashboard/
    layout.tsx              /dashboard (shell only)
    page.tsx                /dashboard
    invoices/
      page.tsx              /dashboard/invoices
      [id]/
        page.tsx            /dashboard/invoices/2
        edit/
          page.tsx          /dashboard/invoices/2/edit
```

output

```
Six URLs are reachable: /, /dashboard, /dashboard/invoices,
/dashboard/invoices/2 and /dashboard/invoices/2/edit, plus the dynamic
repeat of the last two for every id.
```

Example 2: a layout shared by a section.

TSX

```tsx
// app/dashboard/layout.tsx
import Link from 'next/link';
import SideNav from './sidenav';

export default function DashboardLayout({ children }: { children: React.ReactNode }) {
  return (
    <div className="flex">
      <SideNav />
      <div className="flex-1">
        <nav className="flex gap-4 p-4">
          <Link href="/dashboard">Home</Link>
          <Link href="/dashboard/invoices">Invoices</Link>
          <Link href="/dashboard/customers">Customers</Link>
        </nav>
        {children}
      </div>
    </div>
  );
}
```

output

```
Every page under /dashboard shows the same navigation and sidebar, and clicking
between them does not remount the shell, so an open menu stays open.
```

Example 3: a dynamic segment.

TSX

```tsx
// app/dashboard/invoices/[id]/page.tsx
import { getInvoiceById } from '@/app/lib/data';

export default async function InvoicePage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const invoice = await getInvoiceById(id);
  if (!invoice) return <p>Invoice not found</p>;
  return (
    <main>
      <h1>{invoice.name}</h1>
      <p>{invoice.amount} EUR</p>
    </main>
  );
}
```

output

```
Visiting /dashboard/invoices/2 renders the invoice with id 2. In version 14 the
same component read `params.id` directly, which is the most common breakage
when upgrading.
```

Example 4: prerendering a fixed list of dynamic segments.

TSX

```tsx
// app/blog/[slug]/page.tsx
import { getPost } from '@/app/lib/data';

export async function generateStaticParams() {
  const posts = await getPost('all');
  return posts.map((post) => ({ slug: post.slug }));
}

type Props = { params: Promise<{ slug: string }> };

export default async function PostPage({ params }: Props) {
  const { slug } = await params;
  return <article>{getPost(slug)}</article>;
}
```

output

```
`npm run build` reports one prerendered page per slug returned by the
function. URLs outside the list are rendered on demand, unless the segment is
marked dynamic.
```

Example 5: a route handler.

TypeScript

```ts
// app/api/invoices/route.ts
import { NextResponse } from 'next/server';
import { getInvoices } from '@/app/lib/data';

export async function GET() {
  const invoices = await getInvoices();
  return NextResponse.json(invoices);
}
```

shell

```
curl -s http://localhost:3000/api/invoices | head -c 120
```

output

```
[{"id":"1","name":"Acme","amount":250},{"id":"2","name":"Globex","amount":480}]
```

The same folder may also export `POST`, `PATCH` and `DELETE`; the request object is a `NextRequest`, and the response may be created with `NextResponse.json`.

Example 6: route groups.

output

```
app/
  (marketing)/
    layout.tsx        a landing shell with a footer
    about/page.tsx    /about
    pricing/page.tsx  /pricing
  (shop)/
    layout.tsx        a shop shell with a basket
    cart/page.tsx     /cart
```

output

```
The two sections keep separate layouts while their URLs stay clean: /about,
/pricing and /cart, with no extra folder name in the path.
```

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Expecting a folder to be a route	the URL returns not-found	add a `page.tsx` or `route.ts`
Reading `params.id` without `await` in version 15	you read `undefined`	`const { id } = await params`
Two `page.tsx` files for one URL	the build reports a conflict	keep one page per segment, use groups
Putting data logic in a layout	it runs for every child	fetch in the page or in a shared helper
Reaching for `dynamic = 'force-dynamic'` first	static pages become slow	let the build decide, then check the output

Mistake	Why it hurts	Best practice
A `layout.tsx` that fetches unique data per child	impossible by design	keep layouts generic, fetch in pages

4.5 Practice exercises

1. Which file makes the folder `app/blog/` reachable at `/blog`?
2. What does a folder named `app/blog/[slug]/` give you, and how do you read the value in version 15?
3. What is the difference between `layout.tsx` and `template.tsx` in the same folder?
4. How many `page.tsx` files can exist for one URL, and what happens if two collide?
5. Which file exports an HTTP `GET` function inside `app/api/`?

Solutions

1. `app/blog/page.tsx`. Without it the folder can hold shared layouts but not answer for `/blog`.
2. A dynamic segment whose value is available as `params.slug`; in version 15 the component awaits it: `const { slug } = await params;`.
3. Both wrap the children, but a layout persists across navigations inside its segment while a template is remounted on every navigation.
4. One. Two files matching the same route make the build or the dev server report a conflict.
5. `app/api/[...]/route.ts`.

4.6 Mini test

1. Which file turns a folder into a reachable route? A) `layout.tsx` B) `page.tsx` C) `template.tsx` D) `loading.tsx`
2. What does the folder name `app/shop/[id]/` add to the URL? A) the literal text `[id]` B) a dynamic value read from `params.id` C) a query string D) nothing
3. True or false: a route group `(shop)` appears in the URL.
4. Which helper prerenders a known list of dynamic segments? A) `generateStaticParams` B) `generateMetadata` C) `getStaticPaths` D) `revalidatePath`
5. In Next.js 15, what type does the `params` prop of a page have? A) a plain object B) a promise C) a function D) a query object
6. Which file exports `GET` and `POST` for an endpoint instead of a page? A) `app/api/route.tsx` B) `route.ts` C) `handlers.ts` D) `api.tsx`
7. What happens when a layout refuses to render? A) only the layout is replaced by an error boundary B) the whole segment, including its children, shows an error C) the page renders without the layout D) nothing visible changes

Answer key and explanations

1. B - `page.tsx` is what makes a segment public; the other files only add behaviour.

2. B - the folder is a placeholder, and the real value is read from `params`.
3. False - a route group only organises files; the parentheses never reach the browser.
4. A - `generateStaticParams` returns the list of known values at build time.
5. B - version 15 turned route props into promises to support streaming data access.
6. B - `route.ts` exports one function per HTTP verb.
7. B - an error in a layout replaces the whole segment, because the children are rendered inside it.

Chapter 5 - Navigating between pages with the Link component

5.1 Summary

- `<Link href="/invoices">` renders a normal anchor, but intercepts the click and navigates without reloading the page.
- A client navigation fetches only the server components of the target segment, so a shared layout is neither remounted nor re-rendered from scratch.
- In production the framework prefetches linked routes, which is why a click usually feels instant.
- `usePathname()` from `next/navigation` reports the active URL, and it is what a nav item uses to highlight itself.
- `useRouter()` from `next/navigation` exposes `push`, `replace`, `refresh` and `back` for navigation that happens from code instead of from a link.
- The `replace`, `scroll` and `prefetch` props tune history, scroll position and prefetching per link.
- Any component that calls those hooks must be a client component, so an active nav item lives in its own small island.
- A server component that needs to move the user somewhere calls `redirect()` from `next/navigation`, which is a function rather than a hook.
- Because layouts are not remounted, state kept in a layout survives navigation between its pages.

5.2 Concepts

What a link does

Step	What happens
1	The user clicks the anchor
2	The framework reads the route for that URL from the cache, or fetches it
3	Only the changed server components are re-rendered on the server

Step	What happens
4	The browser swaps the new content in and keeps component state in layouts
5	History is updated, so the back button works as expected

A plain anchor, by contrast, throws away the whole document and downloads the application again.

Hooks and helpers for navigation

Import	Kind	Purpose
`Link` from `next/link`	component	declarative navigation
`useRouter` from `next/navigation`	hook, client only	`push`, `replace`, `refresh`, `back`
`usePathname` from `next/navigation`	hook, client only	the current URL path
`useSearchParams` from `next/navigation`	hook, client only	the current query string
`useParams` from `next/navigation`	hook, client only	the dynamic params of the current segment
`redirect` from `next/navigation`	function	send the user to another route from server code

Link options

Prop	Effect
`href`	the target, a string or an object built with the `href` helper
`replace`	replace the current history entry instead of pushing one
`scroll`	set to `false` to keep the scroll position after navigating
`prefetch`	`null` lets the framework decide, `true` forces it, `false` disables it
`passHref`	forward the `href` to a custom child component
`legacyBehavior`	old anchor semantics from the Pages Router era

Rendering the whole payload, or the shell

The automatic prefetch setting fetches the static shell of a route: the layouts and anything above the nearest `loading.tsx` boundary. Dynamic data inside the page still loads when the route is visited, so the first paint is fast and the fresh values arrive a moment later.

5.3 Examples

Example 1: a navigation bar.

TSX

```tsx
// app/ui/nav-links.tsx
'use client';

import Link from 'next/link';
import { usePathname } from 'next/navigation';

const links = [
  { href: '/dashboard', label: 'Home' },
  { href: '/dashboard/invoices', label: 'Invoices' },
  { href: '/dashboard/customers', label: 'Customers' },
];

export default function NavLinks() {
  const pathname = usePathname();
  return (
    <nav className="flex gap-4">
      {links.map((link) => {
        const active = pathname === link.href;
        return (
          <Link
            key={link.href}
            href={link.href}
            className={active ? 'text-blue-700 font-bold' : 'text-blue-500'}
          >
            {link.label}
          </Link>
        );
      })}
    </nav>
  );
}
```

output

```
Clicking "Invoices" swaps the page without a document reload, and the visited
link turns bold because `usePathname` now matches its href.
```

Example 2: navigation from code.

TSX

```tsx
// app/ui/logout-button.tsx
'use client';

import { useRouter } from 'next/navigation';

export default function LogoutButton() {
  const router = useRouter();
  return (
    <button onClick={() => router.replace('/login')}>
      Log out
    </button>
  );
}
```

output

```
The user is taken to /login, and the replaced history entry means the back
button does not return to the page that was just left.
```

Example 3: refreshing server-rendered data after an action.

TSX

```tsx
'use client';

import { useRouter } from 'next/navigation';

export function RefreshButton() {
  const router = useRouter();
  return <button onClick={() => router.refresh()}>Refresh list</button>;
}
```

output

```
The router asks the server for the current route again and re-renders the
server components, while keeping the browser state of client components.
```

Example 4: redirecting from a server component.

TSX

```tsx
// app/dashboard/page.tsx
import { redirect } from 'next/navigation';
import { getSession } from '@/app/lib/auth';

export default async function DashboardPage() {
  const session = await getSession();
  if (!session) redirect('/login');
  return <h1>Welcome back</h1>;
}
```

output

```
A visitor without a session never sees the page; the browser is redirected to
/login instead. The redirect is thrown, so nothing after it in the component
runs.
```

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Using a plain anchor for internal links	the whole app reloads on every click	use `<Link>`
Calling `usePathname` in a server component	a hook error at build or render time	move the active-state logic into a client island
Navigating with `window.location`	loses client-side routing entirely	use `router.push` or `<Link>`
Wrapping a redirect in `try`/`catch` that swallows errors	the redirect is thrown and gets eaten	let it bubble, or rethrow anything you catch
Prefetching every dynamic route eagerly	wasted bandwidth on large pages	leave the default, or set `prefetch={false}`
Storing the active path in server state	the hook is cheaper and always correct	read it from `usePathname`

5.5 Practice exercises

1. Which component performs client-side navigation, and which module provides the hooks for the same job?
2. Why does component state inside `app/dashboard/layout.tsx` survive a click on a link to another dashboard page?
3. How do you move a user to `/login` from inside a server component?
4. Which call asks the server for fresh data without a full navigation?
5. A nav item must highlight the current page. Which hook reports the current path?

Solutions

1. `<Link>` from `next/link` for declarative navigation; `useRouter` and `usePathname` from `next/navigation` for programmatic navigation.
2. Because a client navigation only replaces the parts of the tree that changed; the layout stays mounted, so its state and DOM survive.
3. Call `redirect('/login')` from `next/navigation`, which throws and stops rendering the current tree.
4. `router.refresh()` from the `useRouter()` hook.
5. `usePathname()`, in a component marked `"use client"`.

5.6 Mini test

1. Which module provides `usePathname`? A) `next/link` B) `next/navigation` C) `next/router` D) `next/app`
2. True or false: `<Link>` still renders a real anchor element in the markup.
3. What does `router.replace('/login')` do differently from `push`? A) nothing B) it replaces the current history entry C) it blocks navigation D) it prefetches the route
4. Which of the following may not be used inside a server component? A) `redirect` B) `usePathname` C) `await params` D) a database query
5. A shared layout exists between two linked pages. What happens to it during navigation? A) it is remounted and loses state B) it stays mounted C) it is re-created from scratch on the server D) it is deleted
6. What does `scroll={false}` do? A) disables scrolling in the page B) keeps the scroll position after navigating C) scrolls to the top before navigating D) scrolls to the error boundary
7. Which function is called to re-fetch the current route's server components? A) `router.reload()` B) `router.refresh()` C) `router.replace()` D) `window.location.reload()`

Answer key and explanations

1. B - the App Router hooks live in `next/navigation`.
2. True - the anchor is rendered in the markup, and the click handler improves it.
3. B - the entry is replaced, so the back button skips it.

4. B - hooks need the client; `redirect`, `params` and queries all work on the server.
5. B - only the changed segments are swapped, so the layout is preserved.
6. B - useful for searches, where jumping to the top would be distracting.
7. B - `refresh` re-renders the server components of the current route.

Chapter 6 - Data access and fetching in server components

6.1 Summary

- An `async` server component is the natural place to fetch data: it awaits the result and returns markup, with no state and no effect involved.
- Keeping every query in one data module keeps the components readable and makes the storage engine replaceable.
- In version 15 `fetch` does not cache by default, so a request re-runs for every render unless you opt in.
- `fetch(url, { next: { revalidate: 60 } })` caches for a number of seconds, and `{ next: { tags: ['invoices'] } }` caches until the tag is invalidated.
- Fetching two resources in parallel with `Promise.all` removes the waterfall that two sequential awaits create.
- `cache` from React memoises a function for the duration of one render pass, which stops a shared layout and a page from repeating the same query.
- Database drivers work directly inside a server component, because that code never reaches the browser.
- A query that would block the page can be moved behind a Suspense boundary so the rest of the route streams first, which Chapter 8 covers.
- Values that belong in the address bar, such as a search term or a page number, should be read from `searchParams` instead of being kept in component state.

6.2 Concepts

The data module pattern

Layer	Job
Component	asks for data, renders markup
Data module	runs the query, returns plain data
Database or API	stores the records

The component never knows whether the records came from SQL, from a REST endpoint, or from an array in memory. Swap the module, and the components do not change.

Caching options for `fetch`

Call	Behaviour
`fetch(url)`	no caching in version 15
`fetch(url, { cache: 'force-cache' })`	cache indefinitely, no revalidation
`fetch(url, { next: { revalidate: 60 } })`	cache for 60 seconds, then re-fetch once
`fetch(url, { next: { tags: ['invoices'] } })`	cache until `revalidateTag('invoices')` runs
`fetch(url, { cache: 'no-store' })`	the explicit form of the default

Versions before 15 cached GET requests by default, so a tutorial that omits the options and still expects static output is describing version 14.

Sequential and parallel fetching

output

```
sequential:  invoices 120 ms, then latest 120 ms  -> 240 ms before paint

parallel:    invoices  120 ms
             latest   120 ms                    -> 120 ms before paint
```

`Promise.all` starts both requests, then waits for the slower one. It is the cheapest performance win in a server component.

Request deduplication with `cache`

`cache(fn)` from React keeps one promise per key for the current render, so two components asking for the same invoice cause one query:

Call	Queries run
`getInvoices()` called twice in a tree without `cache`	two
`cachedGetInvoices()` called twice in a tree with `cache`	one

6.3 Examples

Example 1: a data module with an in-memory store.

TypeScript

```ts
// app/lib/data.ts
export type Invoice = {
  id: string;
  name: string;
  amount: number;
  date: string;
};

const invoices: Invoice[] = [
  { id: '1', name: 'Acme', amount: 250, date: '2026-01-14' },
  { id: '2', name: 'Globex', amount: 480, date: '2026-02-02' },
];

export async function getInvoices(): Promise<Invoice[]> {
  await new Promise((resolve) => setTimeout(resolve, 120));
  return invoices;
}

export async function getInvoiceById(id: string): Promise<Invoice | undefined> {
  return invoices.find((invoice) => invoice.id === id);
}
```

output

```
The artificial delay stands in for a database round trip, which makes the
streaming examples of Chapter 8 behave like the real thing.
```

The same module against a database replaces the body of the function:

TypeScript

```ts
export async function getInvoices() {
  return sql`SELECT id, name, amount, date FROM invoices ORDER BY date DESC`;
}
```

Example 2: a page that reads the module.

TSX

```tsx
// app/invoices/page.tsx
import { getInvoices } from '@/app/lib/data';

export default async function InvoicesPage() {
  const invoices = await getInvoices();
  return (
    <main>
      <h1>Invoices</h1>
      <p>{invoices.length} records</p>
      <ul>
        {invoices.map((invoice) => (
          <li key={invoice.id}>
            {invoice.name}: {invoice.amount} EUR
          </li>
        ))}
      </ul>
    </main>
  );
}
```

output

```
Invoices
2 records
Acme: 250 EUR
Globex: 480 EUR
```

Example 3: a sequential waterfall and its fix.

TSX

```tsx
// before: the second await starts only after the first one settles
const invoices = await getInvoices();
const latest = await getLatestInvoice();
```

TSX

```tsx
// after: both requests run at the same time
const [invoices, latest] = await Promise.all([getInvoices(), getLatestInvoice()]);
```

output

```
Both versions return the same data. On a slow connection the first takes the
sum of both latencies, the second takes the longer of the two.
```

Example 4: caching a remote request with a tag.

TypeScript

```ts
export async function getRates() {
  const response = await fetch('https://api.example.com/rates', {
    next: { tags: ['rates'] },
  });
  return response.json();
}
```

output

```
The first request hits the network; later renders reuse the stored response
until some code calls revalidateTag('rates'), after which the next request
refreshes the cache entry.
```

Example 5: memoising a shared query with `cache`.

TypeScript

```ts
// app/lib/data.ts
import { cache } from 'react';

export const getInvoicesCached = cache(async () => {
  return sql`SELECT id, name, amount FROM invoices`;
});
```

output

```
A layout and its page can both call getInvoicesCached() and the query runs
once. The memoisation lives for one request, so the next request starts fresh.
```

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Fetching in `useEffect` what the server can fetch	extra latency and a loading state the user sees	await in the server component
Two awaits in a row	the page waits for the sum of both	`Promise.all`
Assuming `fetch` is cached in version 15	every request goes to the network	state the caching intent explicitly
Querying the database inside a layout	the query runs for every child route	fetch in the page, or memoise with `cache`
Passing a database row object straight to the client	functions and circular references cannot be serialised	map it to plain data
Reading a secret in a client component	the secret is bundled and exposed	keep queries on the server

6.5 Practice exercises

1. Which kind of component can `await` a database query, and why?
2. What are the two ways to opt a `fetch` call into caching, and what do they do differently?
3. Rewrite two sequential awaits so they start at the same time.
4. What problem does `cache` from React solve, and how long does its answer live?
5. Why should a search term be read from `searchParams` rather than stored in state?

Solutions

1. A server component, because its code runs only on the server and never ships to the browser.
2. `next: { revalidate: n }` keeps the response for a number of seconds, and `next: { tags: [...] }` keeps it until a named tag is invalidated. The first is time-based, the second is event-based.
3. Wrap them: `const [a, b] = await Promise.all([first(), second()]);`
4. It prevents the same query from running twice during one render pass, for example when a layout and a page both need the list. Its answer lasts for the current request only.
5. Because the address bar is the only place a user can bookmark, share or reload a search, and because the server can read it to return the matching markup.

6.6 Mini test

1. Which of these runs in the browser and may not touch the database? A) a server component B) a client component C) an `async` page marked with no directive D) a route handler
2. What happens to a plain `fetch(url)` in Next.js 15? A) it is cached forever B) it is cached for 30 seconds C) it is not cached D) it throws

3. Which helper removes a fetch waterfall? A) `Promise.all` B) `Promise.race` C) `cache` D) `revalidatePath`
4. What does `cache` from React return? A) a memoised version of a function B) a cached HTTP response C) a server action D) a promise
5. How long does a `cache` memo last? A) forever B) 30 seconds C) until the process restarts D) for the duration of one render pass
6. True or false: a database driver may be imported by a page component in the App Router.
7. Which prop carries the query string of the current URL into a page component? A) `params` B) `searchParams` C) `query` D) `router.query`

Answer key and explanations

1. B - a client component runs in the browser, where no database credentials exist.
2. C - version 15 changed the default from cached to uncached.
3. A - it starts the requests together and waits for the slower one.
4. A - it wraps a function so repeated calls during one render share a result.
5. D - the memo is per request, so the next render queries again.
6. True - server components can use server-only packages directly.
7. B - `searchParams` is the prop, and in version 15 it is a promise.

Chapter 7 - Static, dynamic rendering and revalidation

7.1 Summary

- Static rendering prerenders the HTML at build time and serves the same bytes to every visitor, which makes the first response fast and cheap.
- Dynamic rendering runs the component for each request, which is what data that differs per user or changes constantly needs.
- A route becomes dynamic as soon as it touches a dynamic API such as `cookies()`, `headers()`, `searchParams`, `draftMode()` or, in version 15, `connection()`.
- Route segment config exports (`dynamic`, `revalidate`, `dynamicParams`, `fetchCache`, `runtime`) declare the strategy in code instead of relying on inference.
- `export const revalidate = 3600` keeps a static route in cache and regenerates it at most once an hour, the behaviour people call incremental static regeneration.
- After a mutation you invalidate what changed with `revalidatePath` for a route or `revalidateTag` for a tagged set of data.
- `fetch` caching and route-level invalidation answer different questions: one is about a single response, the other about a whole rendered page.
- The build output labels every route as static or dynamic, so the fastest way to check an assumption is to read the route table it prints.

- In version 15 a `fetch` without options is not cached, so a page built only on uncached data is normally rendered per request.

7.2 Concepts

The two rendering modes

	Static	Dynamic
Rendered	once, at build time	on every request
Content	identical for everyone	may differ per user
Cache	served from the build output	none
Typical data	public lists, documentation, marketing pages	dashboards, search results, sessions
Cost	almost nothing at runtime	a render and the data queries per visit

The dynamic APIs

Using any of these makes the route dynamic:

API	Import	What it exposes
`cookies()`	`next/headers`	the request cookies, async in version 15
`headers()`	`next/headers`	the request headers, async in version 15
`searchParams` in a page	the component props	the query string, a promise in version 15
`draftMode()`	`next/headers`	preview mode state
`connection()`	`next/server`	waits until a request arrives, replacing `unstable_noStore()`

The escape hatch, when you want a static page that reads data, is `export const dynamic = 'force-static'`. The opposite, `force-dynamic`, renders per request no matter what the code looks like.

Segment configuration

Export	Values	Meaning
`dynamic`	`'auto'`, `'force-static'`, `'force-dynamic'`, `'error'`	the rendering strategy for the segment
`revalidate`	a number of seconds, `false`	how long the prerendered output may be reused
`dynamicParams`	`true`, `false`	serve paths outside `generateStaticParams` or return 404

Export	Values	Meaning
`fetchCache`	`'auto'`, `'force-cache'`, `'only-cache'`	default fetch behaviour for the segment
`runtime`	`'nodejs'`, `'edge'`	where the route executes

Invalidation

Function	Import	Effect
`revalidatePath('/invoices')`	`next/cache`	discard the rendered output for that route
`revalidatePath('/invoices', 'page')`	`next/cache`	target a specific page instead of the whole subtree
`revalidateTag('invoices')`	`next/cache`	discard every entry tagged `invoices`
`revalidatePath` inside an action	-	the usual follow-up to a successful mutation

Tagging starts at the `fetch` call: `fetch(url, { next: { tags: ['invoices'] } })`.

7.3 Examples

Example 1: a page that stays static.

TSX

```tsx
// app/blog/page.tsx
export const revalidate = 3600;

export default async function BlogPage() {
  const posts = await fetch('https://api.example.com/posts', {
    next: { revalidate: 3600 },
  }).then((r) => r.json());
  return (
    <ul>
      {posts.map((post: { id: string; title: string }) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

output

```
 ✓ Compiled successfully
Route (app)                 Size  First Load JS
┌ ● /blog                 132 B         102 kB
│   ISR: revalidate every 3600 seconds
```

Example 2: a page that must be dynamic.

TSX

```tsx
// app/dashboard/page.tsx
import { cookies } from 'next/headers';

export default async function DashboardPage() {
  const jar = await cookies();
  const theme = jar.get('theme')?.value ?? 'light';
  return <h1>Your theme is {theme}</h1>;
}
```

output

```
 ✓ Compiled successfully
Route (app)                Size  First Load JS
┌ ƒ /dashboard             132 B         102 kB
λ  (Server)  renders dynamically on every request
```

The marker next to the route changes from a filled circle to a lambda, which is the build telling you the page is rendered per request.

Example 3: forcing a mode explicitly.

TSX

```tsx
// app/report/page.tsx
export const dynamic = 'force-static';

export default async function ReportPage() {
  const total = await getTotalRevenue();
  return <p>Total revenue: {total}</p>;
}
```

output

```
The page is prerendered even though it awaits data, and every visitor receives
the same value until the route is revalidated or the site is rebuilt.
```

Example 4: time-based and on-demand invalidation together.

TSX

```tsx
// app/invoices/page.tsx
export const revalidate = 3600;

export default async function InvoicesPage() {
  const invoices = await getInvoices();
  return <InvoiceTable invoices={invoices} />;
}
```

TypeScript

```ts
// app/lib/actions.ts
'use server';

import { revalidatePath, revalidateTag } from 'next/cache';

export async function createInvoice(formData: FormData) {
  const name = String(formData.get('name') ?? '');
  const amount = Number(formData.get('amount'));
  await sql`INSERT INTO invoices (name, amount) VALUES (${name}, ${amount})`;
  revalidatePath('/invoices');
  revalidateTag('invoices');
}
```

output

```
The `sql` helper is the project's query client, as in Chapter 6. The route
refreshes at most once an hour on its own, and immediately after a new invoice
is created, whichever comes first.
```

Example 5: only the paths you listed.

TSX

```tsx
// app/invoices/[id]/page.tsx
export const dynamicParams = false;

export async function generateStaticParams() {
  return [{ id: '1' }, { id: '2' }];
}
```

output

```
Requests for /invoices/1 and /invoices/2 are prerendered. /invoices/9 answers
with a not-found response instead of being rendered on demand.
```

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Reading a session in a marketing page	a static page becomes dynamic	move the check into the dashboard
Forgetting to invalidate after a mutation	users see stale rows	`revalidatePath` in the action
Using `force-dynamic` everywhere	every request pays full price	let the framework infer, then check the build
Revalidating the whole site	needless builds and cache misses	revalidate the affected path or tag
Setting a very short `revalidate`	you pay for dynamic rendering anyway	pick a realistic interval
Assuming the version 14 fetch cache	data silently never updates	state the caching intent per request

7.5 Practice exercises

1. Which three APIs make a route dynamic without any configuration?
2. What does `export const revalidate = 60` do?
3. Which two functions invalidate cached output, and when do you prefer each?
4. What does `dynamicParams = false` change for a dynamic segment?
5. How can you tell from the terminal which routes are static and which are dynamic?

Solutions

1. `cookies()`, `headers()` and `searchParams`, plus `draftMode()` and `connection()` in version 15.
2. It tells the framework that the prerendered output of the segment may be reused for 60 seconds before it is generated again.
3. `revalidatePath` when you know the route that must refresh, and `revalidateTag` when a set of unrelated routes shares the same data.
4. Requests for values outside `generateStaticParams` return a not-found response instead of being rendered on demand.
5. Run `npm run build` and read the route table: a circle marks static output, a lambda marks a dynamic route, and filled circles carry extra notes such as the revalidate interval.

7.6 Mini test

1. Which of these makes a route dynamic? A) `revalidate = 3600` B) `await cookies()` C) `generateStaticParams` D) `force-static`
2. What does `export const dynamic = 'force-dynamic'` do? A) prerenders at build time B) renders on every request C) disables caching of `fetch` D) returns 404 for unknown params
3. Which export declares how long a prerendered page may be reused? A) `dynamic` B) `revalidate` C) `runtime` D) `dynamicParams`
4. Which function invalidates everything tagged `invoices`? A) `revalidatePath('invoices')` B) `revalidateTag('invoices')` C) `unstable_cache` D) `cache.clear()`
5. True or false: in Next.js 15 a `fetch` without options is cached for 30 seconds.
6. What does a lambda marker next to a route in the build output mean? A) the route errored B) the route is dynamic C) the route is an API route D) the route is in a route group
7. Where is `connection()` imported from in version 15? A) `next/headers` B) `next/cache` C) `next/server` D) `next/navigation`

Answer key and explanations

1. B - cookies differ per request, so the page cannot be shared.
2. B - it forces dynamic rendering regardless of what the component uses.
3. B - `revalidate` is the time-to-live of the prerendered output.
4. B - tags invalidate every fetch and route that shares the label.
5. False - version 15 removed the default `fetch` cache.

6. B - the lambda marks requests that are rendered per request.
7. C - `connection()` came from the server package as the replacement for `unstable_noStore`.

Chapter 8 - Streaming and loading states

8.1 Summary

- Streaming sends the shell of a route to the browser immediately and fills the slow parts as their data arrives.
- Adding a `loading.tsx` file wraps that segment's children in a Suspense boundary, with your fallback shown meanwhile.
- A Suspense boundary shows its fallback until every promise inside it resolves, then swaps in the real content in one step.
- Boundaries belong around independent queries, not around the whole page, so a slow table does not hold up the navigation.
- Skeletons are the recommended fallback for structured content, because they keep the layout stable while the data lands.
- Streaming moves work off the critical path, so the time to first byte and the first contentful paint no longer wait for the slowest query.
- A route without any boundary renders nothing until its slowest query finishes, which is the symptom users read as a hanging page.
- Content that is personalized can still stream into a static shell; the experimental name for that combination is partial prerendering and it depends on your minor version.
- During navigation a pending indicator can be built from `useTransition`, or taken from the dedicated link hook in newer releases.

8.2 Concepts

What the browser receives

output

```
without streaming                with streaming
-----------------                -------------
request                          request
  |                                |
query invoices (400 ms)           shell + skeleton  (arrives in ~50 ms)
  |                                |
paint the whole page              query invoices (400 ms)
                                     |
                                  swap in the real table
```

The second column paints something useful in the first round trip and improves it without hiding the delay.

The three levels of control

Mechanism	Scope	Where it is declared
`loading.tsx`	the whole segment below the folder	a file in the segment
`<Suspense boundary="..." fallback={...}>`	one component or group of components	inside the component tree
No boundary anywhere	the entire route waits for the slowest query	the default

Replacing the file-based boundary with your own `<Suspense>` is the normal next step once a route has more than one slow region.

Writing a fallback

Content	Good fallback
A table	grey bars in the shape of rows
A chart	a box of the same height
A heading with numbers	the heading, with placeholder digits
Anything else	a short line of text, not a full-page spinner

Match the size, or the content will jump when it replaces the fallback.

8.3 Examples

Example 1: the file-based boundary.

TSX

```tsx
// app/invoices/loading.tsx
export default function Loading() {
  return (
    <div className="animate-pulse">
      <div className="h-6 w-40 rounded bg-gray-200" />
      <div className="mt-4 h-24 rounded bg-gray-200" />
    </div>
  );
}
```

output

```
Opening /invoices shows the grey skeleton for as long as the query takes, then
the table appears in its place. The navigation and the header never disappear,
because they live above the boundary.
```

Example 2: two independent regions.

TSX

```tsx
// app/dashboard/page.tsx
import { Suspense } from 'react';
import LatestInvoices, { LatestInvoicesSkeleton } from './latest-invoices';
import RevenueChart, { RevenueChartSkeleton } from './revenue-chart';

export default function DashboardPage() {
  return (
    <main>
      <h1>Dashboard</h1>
      <Suspense fallback={<RevenueChartSkeleton />}>
        <RevenueChart />
      </Suspense>
      <Suspense fallback={<LatestInvoicesSkeleton />}>
        <LatestInvoices />
      </Suspense>
    </main>
  );
}
```

output

```
The chart takes 800 ms and the list 300 ms. The list therefore appears first,
while the chart area keeps its skeleton for the remaining 500 ms.
```

Example 3: a component whose promise the boundary hides.

TSX

```tsx
// app/dashboard/revenue-chart.tsx
export default async function RevenueChart() {
  const data = await getRevenue(); // 800 ms
  return <Chart data={data} />;
}

export function RevenueChartSkeleton() {
  return <div className="h-48 rounded bg-gray-200" />;
}
```

output

```
An `async` server component inside a Suspense boundary can simply await: the
boundary is what turns the wait into a streaming swap instead of a blocked page.
```

Example 4: the same page with no boundary.

TSX

```tsx
// app/dashboard/page.tsx
export default async function DashboardPage() {
  const chart = await getRevenue(); // 800 ms
  const latest = await getLatestInvoices(); // 300 ms
  return <Dashboard chart={chart} latest={latest} />;
}
```

output

```
Nothing is painted for more than a second. Adding the boundaries of Example 2
is the whole fix.
```

Example 5: a pending state during navigation.

TSX

```tsx
// app/ui/nav-pending.tsx
'use client';

import { useTransition } from 'react';

export function useNavPending() {
  const [pending, startTransition] = useTransition();
  return { pending, startTransition };
}
```

output

```
The flag turns true while a navigation is in flight, which is enough to show a
thin progress bar or a dimmed overlay. Newer minor versions also ship a hook
that reports the pending state of a link, so check the release you run.
```

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
One boundary around everything	the page still waits for the slowest part	wrap each slow region on its own
Fallbacks of a different size	the layout jumps when content arrives	match the height of the real content
A spinner for a dashboard	repeated flashes feel broken	use skeletons that mirror the content
Fetching sequentially then wondering why loading is slow	the delays add up	fetch in parallel inside one boundary
Hiding a slow query behind a forever skeleton	users think the app is stuck	keep waits short, and reconsider the query
Removing `loading.tsx` to "fix" a jump	the whole route now blocks	fix the fallback size instead

8.5 Practice exercises

1. Which file adds a streaming boundary to a whole route segment, and what does it render?
2. Why should each slow region get its own boundary?
3. A table's skeleton is 200 px tall while the table is 300 px. What does the user see when the data arrives?
4. How does streaming change the time to first byte?
5. What does an `async` server component inside a Suspense boundary need in order to stream?

Solutions

1. `loading.tsx`; it renders the fallback shown while the segment's children are still waiting.

2. Because a boundary is only as fast as its slowest child; a single wrapper would delay the fast regions behind the slow one.
3. The content shifts down by 100 px at the moment of the swap, which reads as an unstable page.
4. The shell is sent in the first round trip, so the first bytes no longer wait for the slowest query in the page.
5. Nothing special: it just awaits its data, and the boundary takes care of the rest.

8.6 Mini test

1. Which file provides the fallback for a whole route segment? A) `error.tsx` B) `loading.tsx` C) `suspense.tsx` D) `fallback.tsx`
2. True or false: a Suspense boundary renders its fallback until every promise inside it resolves.
3. What is the recommended fallback for a table? A) a blank page B) a skeleton with rows of the same size C) an error message D) a modal dialog
4. Two regions take 300 ms and 800 ms and each has its own boundary. Which arrives first? A) the 800 ms region B) they arrive together C) the 300 ms region D) neither streams
5. What happens to a route with no boundary at all? A) it loads instantly B) nothing is painted until the slowest query finishes C) it throws D) it is always static
6. Which hook reports that a navigation is in progress? A) `useState` B) `useTransition` C) `useEffect` D) `useRef`
7. What is the experimental name for a static shell with streamed dynamic holes? A) incremental static regeneration B) partial prerendering C) client-side rendering D) incremental streaming

Answer key and explanations

1. B - `loading.tsx` is the file convention for the segment's Suspense fallback.
2. True - the boundary resolves as a unit, then swaps in the content.
3. B - a shape-matched skeleton avoids layout shift while the data loads.
4. C - the faster region streams in as soon as its query resolves.
5. B - without a boundary the response starts only when rendering is complete.
6. B - `useTransition` exposes the pending flag during a navigation.
7. B - a static shell plus streamed holes is what partial prerendering means, and it remains experimental.

Chapter 9 - Search and pagination

9.1 Summary

- The URL is the natural home for search state, because it is shareable, bookmarkable and restores the same result with the back button.
- A page component reads the query string from `searchParams`, which in version 15 arrives as a promise and is awaited.
- A client component reads and writes the same values with `useSearchParams`, `usePathname` and `router.replace`.
- Updating the address on every keystroke hammers the server, so the input is debounced and only the last value reaches the URL.
- In a statically rendered route, a component that reads the query string needs a Suspense boundary, otherwise the segment gives up prerendering.
- Pagination in SQL is `LIMIT` and `OFFSET`, with a `COUNT(*)` query to learn the number of pages.
- `new URLSearchParams(searchParams)` copies the current values, so only the parameter you intend to change is rewritten.
- A small helper that builds the page links keeps the active search term in the address while the page number changes.
- Search and pagination interact: a new query changes the total number of pages, so the page number should be reset to one.

9.2 Concepts

The shape of the query string

output

```
/invoices?query=acme&page=3
  |            |      |
  |            |      the page number
  |            the search term
   the route
```

Server side the values arrive in `searchParams`; client side the same values are read with `useSearchParams().get('query')`.

Client or server reading

Situation	Read with
Filtering the rows the server renders	`searchParams` in the page component
Keeping a text input in sync with the address	`useSearchParams` in a client component
Changing the page without a round trip	`router.replace` with a rebuilt query string
Sharing or bookmarking the view	the URL does it for free

Debouncing

output

```
keystrokes: a  ac  acm  acme
            |  |   |    |
without debounce: 4 requests, 3 of them useless
with 300 ms debounce: 1 request for "acme"
```

Pagination maths

Expression	Meaning
`LIMIT 10`	page size
`OFFSET (page - 1) * 10`	how many rows to skip
`SELECT count(*) FROM ... WHERE ...`	rows matching the filter
`Math.ceil(count / 10)`	total pages
`page < 1 or page > totalPages`	clamp, or show the empty state

9.3 Examples

Example 1: the server side of search.

TSX

```tsx
// app/invoices/page.tsx
import { getFilteredInvoices } from '@/app/lib/data';
import Pagination from './pagination';

const PER_PAGE = 10;

export default async function InvoicesPage({
  searchParams,
}: {
  searchParams: Promise<{ query?: string; page?: string }>;
}) {
  const sp = await searchParams;
  const query = sp.query ?? '';
  const page = Math.max(1, Number(sp.page) || 1);

  const { invoices, totalPages } = await getFilteredInvoices(query, page, PER_PAGE);

  return (
    <main>
      <h1>Invoices</h1>
      {invoices.length === 0 ? (
        <p>No invoice matches {query}</p>
      ) : (
        <table>
          <tbody>
            {invoices.map((invoice) => (
              <tr key={invoice.id}>
                <td>{invoice.name}</td>
                <td>{invoice.amount}</td>
              </tr>
            ))}
          </tbody>
        </table>
      )}
      <Pagination
        totalPages={totalPages}
        currentPage={page}
        searchParams={{ query }}
      />
    </main>
  );
}
```

output

```
/invoices lists the first ten rows. /invoices?query=acme shows only Acme rows,
and /invoices?page=3 jumps to the third page of the unfiltered list.
```

Example 2: the queries behind the page.

TypeScript

```ts
// app/lib/data.ts
export async function getFilteredInvoices(query: string, page: number, perPage: number) {
  const like = '%' + query + '%';
  const rows = await sql`
    SELECT id, name, amount FROM invoices
    WHERE name ILIKE ${like}
    ORDER BY date DESC
    LIMIT ${perPage} OFFSET ${(page - 1) * perPage}`;
  const total = await sql`
    SELECT count(*)::int AS count FROM invoices
    WHERE name ILIKE ${like}`;
  const count = Number(total[0].count);
  return { invoices: rows, totalPages: Math.max(1, Math.ceil(count / perPage)) };
}
```

output

```
Both statements are parameterised, so a search for the string "100% true"
returns no rows instead of breaking the query. The page count never drops below
one, which keeps the pagination links usable.
```

Example 3: a debounced client search field.

TSX

```tsx
// app/ui/search.tsx
'use client';

import { useEffect, useState } from 'react';
import { usePathname, useRouter, useSearchParams } from 'next/navigation';

export default function Search({ placeholder }: { placeholder: string }) {
  const searchParams = useSearchParams();
  const pathname = usePathname();
  const router = useRouter();
  const [text, setText] = useState(searchParams.get('query') ?? '');

  useEffect(() => {
    const timer = setTimeout(() => {
      const params = new URLSearchParams(searchParams);
      params.set('page', '1');
      if (text) params.set('query', text);
      else params.delete('query');
      router.replace(`${pathname}?${params.toString()}`);
    }, 300);
    return () => clearTimeout(timer);
  }, [text]);

  return (
    <input
      className="border p-2"
      value={text}
      onChange={(event) => setText(event.target.value)}
      placeholder={placeholder}
    />
  );
}
```

output

```
Typing "acme" waits for the 300 ms pause, then the address becomes
/invoices?query=acme&page=1 and the server re-renders the list. Clearing the
field removes the parameter again.
```

Example 4: pagination links that keep the search.

TSX

```tsx
// app/invoices/pagination.tsx
import Link from 'next/link';

type Props = {
  totalPages: number;
  currentPage: number;
  searchParams: { query?: string };
};

export default function Pagination({ totalPages, currentPage, searchParams }: Props) {
  const hrefFor = (page: number) => {
    const params = new URLSearchParams();
    if (searchParams.query) params.set('query', searchParams.query);
    params.set('page', String(page));
    return `/invoices?${params.toString()}`;
  };

  return (
    <nav className="flex gap-2">
      {currentPage > 1 && (
        <Link href={hrefFor(currentPage - 1)}>Previous</Link>
      )}
      <span>
        Page {currentPage} of {totalPages}
      </span>
      {currentPage < totalPages && (
        <Link href={hrefFor(currentPage + 1)}>Next</Link>
      )}
    </nav>
  );
}
```

output

```
While the search is "acme", the next link points to
/invoices?query=acme&page=2. On the last page the forward link disappears.
```

Example 5: reading the query string on the client.

TSX

```tsx
'use client';

import { useSearchParams } from 'next/navigation';

export function ResultCount() {
  const searchParams = useSearchParams();
  const query = searchParams.get('query') ?? '';
  return <p>Showing results for {query || 'everything'}</p>;
}
```

output

```
The component re-renders whenever the address bar changes, without any prop
passing. In a static route it must sit inside a Suspense boundary.
```

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Holding the search term only in state	reload loses it, URLs cannot be shared	write it to the address bar
Updating the URL on every keystroke	a request per character	debounce the input
Building the new query string by hand	parameters are lost or duplicated	start from `new URLSearchParams(searchParams)`
Forgetting to reset the page number	the user sees an empty page 4	set `page=1` when the query changes
Reading query params on the server in a client component	hooks are not available there	keep the URL logic in one client island
Concatenating the search term into SQL	injection and broken queries	parameterise the query

9.5 Practice exercises

1. Why is the URL a better place for a search term than component state?
2. What are the two ways to read the query string, and where does each work?
3. How do you keep the search term while building the link for page 4?
4. What must happen to the page number when the user types a new query?
5. Why is a debounce needed, and what is a reasonable delay?

Solutions

1. Because the URL survives a reload, can be shared and bookmarked, and works with the back button, while state disappears the moment the document is replaced.
2. `searchParams` in a page component on the server, and `useSearchParams` in a client component.
3. Copy the current values with `new URLSearchParams(searchParams)`, then `params.set('page', '4')`, and use the resulting string as the href.
4. It should return to 1, because a new query changes the number of results and therefore the number of pages.
5. Because every keystroke would otherwise trigger a server render; a pause of 200 to 400 ms is a common compromise.

9.6 Mini test

1. Where does a page component read the query string from? A) `params` B) `searchParams` C) `useSearchParams` D) `router.query`
2. In Next.js 15, what type is the `searchParams` prop of a page? A) a plain object B) a promise C) a string D) a map
3. Which client hook reads the current query string? A) `useParams` B) `useSearchParams` C) `usePathname` D) `useRouter`
4. Why is the search input debounced? A) to make typing faster B) to avoid one server render per keystroke C) because the server is single threaded D) to satisfy ESLint
5. What is the SQL for the third page of ten rows? A) `LIMIT 3 OFFSET 10` B) `LIMIT 10 OFFSET 30` C) `LIMIT 10 OFFSET 20` D) `LIMIT 30 OFFSET 10`
6. What does `COUNT(*)` give you in a paginated query? A) the rows of the page B) the rows matching the filter, to derive the page count C) the number of columns D) the size of the result set in bytes
7. A statically rendered route reads `useSearchParams`. What does it need? A) nothing B) a Suspense boundary C) an API route D) `dynamic = 'force-static'`

Answer key and explanations

1. B - the prop carries the query string into the page.
2. B - version 15 turned it into a promise, like `params`.
3. B - `useSearchParams` is the client hook for the query string.
4. B - the pause collapses a burst of keystrokes into one request.
5. C - page 3 skips the first 20 rows and takes the next 10.
6. B - the count divided by the page size gives the number of pages.
7. B - without a boundary the segment cannot be prerendered, and the build tells you so.

Chapter 10 - Mutations with Server Actions

10.1 Summary

- A Server Action is an `async` function marked `"use server"` that the browser may call like an endpoint, without anyone writing that endpoint.
- The directive can sit at the top of a module or on a single function, and in both cases the body runs only on the server.
- When an action is used as the `action` of a form, the framework hands it a `FormData` object whose values are read with `formData.get('name')`.
- After changing data you invalidate what changed with `revalidatePath` or `revalidateTag`, otherwise the page keeps showing the old rows.
- `redirect()` ends the flow by throwing, so it must never be swallowed by a `catch` block that does not rethrow.

- Failures that should reach the user are caught and returned as a plain result object, while unexpected errors bubble up to the error boundary.
- A Server Action is a public entry point: it is reachable by anyone, so the permission check belongs inside the function.
- An action can be called from a form in a server component, from a click handler in a client component, or inside a transition.
- The form that submits to an action works before hydration, so a slow connection still delivers the data.

10.2 Concepts

Where the directive goes

Form	Effect
`'use server';` at the top of the file	every exported async function in the file is an action
`'use server';` on one function	that function alone is an action
importing an action into a client component	the function becomes a remote call, and its arguments must be serialisable

The life of a mutation

output

```
the form is submitted
       |
       v
the browser sends the FormData to the action's endpoint
       |
       v
the action validates, checks the session, writes to the database
       |
       v
revalidatePath / revalidateTag tells the cache what is stale
       |
       v
the server re-renders the affected route and returns fresh HTML
       |
       v
redirect() finishes the flow, if the action calls it
```

Errors and results

Situation	Pattern
Invalid input	return `{ errors: { field: ['message'] } }`
Not authorised	return `{ errors: { form: ['Not authorised'] } }`
Success	`revalidatePath(...)` and optionally `redirect(...)`
Unexpected failure	let it throw; `error.tsx` catches it

Situation	Pattern
After a successful call in a form	read the returned state with `useActionState`

Security

An action is a URL. The fact that only your own form links to it is a convenience, not a rule, so an action re-checks the session and the user's rights before it touches data.

10.3 Examples

Example 1: a module of actions.

TypeScript

```ts
// app/lib/actions.ts
'use server';

import { revalidatePath } from 'next/cache';
import { redirect } from 'next/navigation';

export async function createInvoice(formData: FormData) {
  const name = String(formData.get('name') ?? '');
  const amount = Number(formData.get('amount'));
  await sql`INSERT INTO invoices (name, amount) VALUES (${name}, ${amount})`;
  revalidatePath('/invoices');
  redirect('/invoices');
}

export async function deleteInvoice(formData: FormData) {
  const id = String(formData.get('id') ?? '');
  await sql`DELETE FROM invoices WHERE id = ${id}`;
  revalidatePath('/invoices');
}
```

output

```
Both functions run only on the server. The browser receives a reference it can
call, never the code that reads the database.
```

Example 2: a form that calls the action.

TSX

```tsx
// app/ui/create-form.tsx
import { createInvoice } from '@/app/lib/actions';

export default function CreateForm() {
  return (
    <form action={createInvoice}>
      <label htmlFor="name">Customer</label>
      <input id="name" name="name" />
      <label htmlFor="amount">Amount</label>
      <input id="amount" name="amount" type="number" step="0.01" />
      <button type="submit">Create invoice</button>
    </form>
  );
}
```

output

```
Submitting the form sends the two fields to createInvoice, the invoice is
stored, the cached list is invalidated, and the browser is redirected to
/invoices where the new row is already visible.
```

Example 3: calling an action from a client component.

TSX

```tsx
// app/ui/delete-button.tsx
'use client';

import { useTransition } from 'react';
import { deleteInvoice } from '@/app/lib/actions';

export default function DeleteButton({ id }: { id: string }) {
  const [pending, startTransition] = useTransition();
  return (
    <button
      disabled={pending}
      onClick={() => {
        const data = new FormData();
        data.set('id', id);
        startTransition(() => deleteInvoice(data));
      }}
    >
      {pending ? 'Deleting' : 'Delete'}
    </button>
  );
}
```

output

```
The button shows "Deleting" while the action runs, and the row disappears from
the table once the revalidated route returns.
```

Example 4: returning failures instead of throwing.

TypeScript

```ts
'use server';

import { revalidatePath } from 'next/cache';

export async function createInvoice(formData: FormData) {
  const name = String(formData.get('name') ?? '').trim();
  const amount = Number(formData.get('amount'));
  const errors: { name?: string[]; amount?: string[] } = {};
  if (!name) errors.name = ['Customer is required'];
  if (!Number.isFinite(amount) || amount <= 0) {
    errors.amount = ['Amount must be positive'];
  }
  if (Object.keys(errors).length > 0) return { errors };

  await sql`INSERT INTO invoices (name, amount) VALUES (${name}, ${amount})`;
  revalidatePath('/invoices');
  return { errors: {} };
}
```

output

```
The form stays on screen and shows the messages; nothing is written until both
fields are valid. Chapter 11 wires the returned object into the markup.
```

Example 5: authorising inside the action.

TypeScript

```ts
'use server';

import { revalidatePath } from 'next/cache';
import { getSession } from '@/app/lib/auth';

export async function deleteInvoice(formData: FormData) {
  const session = await getSession();
  if (!session?.user) return { errors: { form: ['Please log in first'] } };
  const id = String(formData.get('id') ?? '');
  await sql`DELETE FROM invoices WHERE id = ${id} AND owner = ${session.user.id}`;
  revalidatePath('/invoices');
  return { errors: {} };
}
```

output

```
The ownership condition is part of the query, so a valid session cannot delete
another user's invoice even if it guesses the id.
```

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Wrapping `redirect` in a `try`/`catch` that swallows everything	the redirect is thrown, so navigation stops	rethrow, or redirect outside the block
Forgetting to revalidate	the user sees the previous list	invalidate the path or the tag on every write
Trusting the form to enforce permissions	actions are public endpoints	check the session and the ownership in the action
Passing a non-serialisable argument to an action	the call fails at the boundary	send plain data or a `FormData`
Throwing raw database errors	the user sees a stack trace	catch, log, and return a safe message
Reading form values without checking the type	`null` or `NaN` reaches the query	convert with `String` and `Number`, then validate

10.5 Practice exercises

1. Which directive turns an async function into a Server Action, and where may it be written?
2. What does the browser receive when a module of actions is exported?
3. Why must a `redirect()` inside a `try` block be handled carefully?
4. Which two calls invalidate cached data after a mutation?

5. An action deletes a row. Which two checks must it perform before the delete runs?

Solutions

1. `"use server"`, either at the top of the module or directly above one function.
2. A reference it may call, never the code itself; the function body stays on the server.
3. Because `redirect` works by throwing a special error, and a `catch` that does not rethrow cancels the navigation.
4. `revalidatePath(...)` for a route and `revalidateTag(...)` for everything carrying that tag.
5. That the user has a session, and that the user owns the row, ideally expressed in the `WHERE` clause of the delete.

10.6 Mini test

1. Which directive marks a Server Action? A) `"use client"` B) `"use server"` C) `"use strict"` D) `export const action = true`
2. What does an action receive when a form submits to it? A) an event object B) a `FormData` C) a JSON body D) an array of strings
3. How does `redirect` end an action? A) it returns `true` B) it throws C) it sets a cookie D) it calls `window.location`
4. Which call invalidates the rendered output of `/invoices`? A) `revalidateTag('/invoices')` B) `revalidatePath('/invoices')` C) `refresh('/invoices')` D) `cache.clear('/invoices')`
5. True or false: a Server Action can be called by a user who found its URL.
6. Which of these may be passed as an argument to a Server Action? A) a function B) a database connection C) a plain object of strings and numbers D) a class instance
7. What does a form bound to an action do before the page has hydrated? A) nothing, it is disabled B) it still submits to the action C) it throws D) it waits for the client bundle

Answer key and explanations

1. B - `"use server"` marks the function as server-only code.
2. B - the framework serialises the form into `FormData` and hands it to the action.
3. B - navigation is performed by throwing a control-flow error, which is why try/catch needs care.
4. B - `revalidatePath` drops the cached route output.
5. True - an action is an endpoint, so the permission check lives inside the function.
6. C - arguments cross the network boundary, so they must be serialisable.
7. B - forms that use actions are progressively enhanced and work without JavaScript.

Chapter 11 - Forms and server-side validation

11.1 Summary

- A form that posts to a Server Action works before the page has hydrated, so it is progressive by default.
- `useActionState` returns the previous state, the wrapped action and a pending flag: `const [state, formAction, pending] = useActionState(action, initialState)`.
- The same helper was called `useFormState` and imported from `react-dom` in React 18; Next.js 15 ships React 19, where it is `useActionState` from `react` with a slightly different signature.
- `useFormStatus` from `react-dom` reports the `pending` state of the nearest enclosing form, which is how a submit button disables itself.
- Validation belongs on the server: everything in the browser is a suggestion, never a guarantee.
- A schema library such as Zod parses the raw form values and returns a structured list of field errors.
- Errors reach the user through `aria-invalid`, `aria-describedby` and an element with an id, which is what makes a screen reader announce them.
- `noValidate` on the form keeps browser tooltips away while the server rules decide, and `required` remains as a hint for the browser and for assistive technology.
- After a failed submit the typed values should stay in the fields, and the first invalid field should receive focus.

11.2 Concepts

The state an action returns

Field	Content
`errors`	an object of field name to a list of messages
`message`	a short summary of the outcome
the previous state	the value the previous call returned, passed as the first argument

The action signature becomes `(previousState, formData) => nextState`, which is why the helper can show the outcome of the last submit.

The two client helpers

Helper	Import	Returns	Where it lives
`useActionState`	`react`	state, wrapped action, pending	the component that owns the form
`useFormStatus`	`react-dom`	`pending`, `data`, `method`, `action`	a child of the form, such as the submit button

`useFormStatus` only reports forms above it in the tree, so the submit button must be rendered inside the `<form>`.

Version note

React	Hook	Import
18	`useFormState`	`react-dom`
19	`useActionState`	`react`

The older form took `(action, initialState)` with an action that accepted only `FormData`; the newer one expects `(previousState, formData)`. Mixing the two produces a form that never submits.

Accessible error reporting

Markup	Purpose
`<label htmlFor="amount">`	names the field for everyone
`aria-invalid={!!errors.amount}`	marks the field as wrong
`aria-describedby="amount-error"`	points at the message element
`id="amount-error"` on a `<p>` or `<div>`	holds the message text
`aria-live="polite"` on the container	announces the message when it appears

11.3 Examples

Example 1: a schema and the action that uses it.

TypeScript

```ts
// app/lib/validation.ts
import { z } from 'zod';

export const InvoiceSchema = z.object({
  customerId: z.string().min(1, { message: 'Please select a customer.' }),
  amount: z.coerce
    .number()
    .gt(0, { message: 'Please enter an amount greater than $0.' }),
  status: z.enum(['open', 'paid'], { message: 'Please choose a status.' }),
});
```

TypeScript

```ts
// app/lib/actions.ts
'use server';

import { revalidatePath } from 'next/cache';
import { InvoiceSchema } from './validation';

export type State = { errors: Record<string, string[]>; message: string | null };

export async function createInvoice(state: State, formData: FormData): Promise<State> {
  const parsed = InvoiceSchema.safeParse({
    customerId: formData.get('customerId'),
    amount: formData.get('amount'),
    status: formData.get('status'),
  });

  if (!parsed.success) {
    return {
      errors: parsed.error.flatten().fieldErrors,
      message: 'Missing fields. Failed to create invoice.',
    };
  }

  const { customerId, amount, status } = parsed.data;
  await sql`INSERT INTO invoices (customer_id, amount, status)
            VALUES (${customerId}, ${amount}, ${status})`;
  revalidatePath('/dashboard/invoices');
  return { errors: {}, message: 'Invoice created.' };
}
```

output

```
Submitting an empty form returns the field errors with the summary "Missing
fields. Failed to create invoice." and nothing is written to the database. A
valid submit returns "Invoice created." and refreshes the list.
```

Zod changed the shape of its flattened errors between major versions; in newer releases the same information is also available from a `flattenError` helper.

Example 2: the form component.

TSX

```tsx
// app/ui/create-form.tsx
'use client';

import { useActionState } from 'react';
import { createInvoice, type State } from '@/app/lib/actions';
import { SubmitButton } from './submit-button';

const initialState: State = { errors: {}, message: null };

export default function CreateForm() {
  const [state, formAction, pending] = useActionState(createInvoice, initialState);
  return (
    <form action={formAction} noValidate>
      <label htmlFor="customer">Customer</label>
      <select id="customer" name="customerId" defaultValue="">
        <option value="" disabled>Select a customer</option>
        <option value="1">Acme</option>
      </select>
      <div id="customer-error" aria-live="polite">
        {state.errors.customerId?.map((error) => <p key={error}>{error}</p>)}
      </div>
      <SubmitButton label="Create invoice" />
      <p aria-live="polite">{state.message}</p>
    </form>
  );
}
```

output

```
The submit button reports its pending state, and the messages below each field
appear only when the server reports a problem.
```

Example 3: the submit button.

TSX

```tsx
// app/ui/submit-button.tsx
'use client';

import { useFormStatus } from 'react-dom';

export function SubmitButton({ label }: { label: string }) {
  const { pending } = useFormStatus();
  return (
    <button type="submit" aria-disabled={pending} disabled={pending}>
      {pending ? 'Saving' : label}
    </button>
  );
}
```

output

```
While the action runs the button reads "Saving" and is disabled, so a double
click cannot create two invoices. The state comes from the form, not from props,
so no wiring is needed.
```

Example 4: an error tied to its input.

TSX

```tsx
<label htmlFor="amount">Amount</label>
<input
  id="amount"
  name="amount"
  type="number"
  step="0.01"
  placeholder="Enter amount"
  defaultValue=""
  aria-describedby="amount-error"
  aria-invalid={Boolean(state.errors.amount)}
/>
<div id="amount-error" aria-live="polite">
  {state.errors.amount?.map((error) => (
    <p key={error} className="text-sm text-red-700">{error}</p>
  ))}
</div>
```

output

```
With an empty amount the markup becomes aria-invalid="true" on the input, and
the paragraph below reads "Please enter an amount greater than $0.".
```

Example 5: an action without a client component.

TSX

```tsx
// app/page.tsx
import { createInvoice } from '@/app/lib/actions';

export default function Page() {
  return (
    <form action={createInvoice}>
      <input name="amount" type="number" />
      <button type="submit">Create</button>
    </form>
  );
}
```

output

```
This simpler form submits to the action without `useActionState`. It works
without JavaScript and shows nothing more than a page navigation in return.
```

11.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Validating only in the browser	the browser copy is optional	validate in the action, and optionally again on the client
Using `useFormState` against React 19	the call never resolves	import `useActionState` from `react`
Rendering the submit button outside the form	`useFormStatus` sees no form	keep the button inside the `<form>`
Clearing the fields after a failed submit	the user types everything again	keep the values, show the errors

Mistake	Why it hurts	Best practice
Error text with no association to the field	screen reader users hear nothing	use `aria-describedby` and an id
Empty messages such as "Invalid input"	the user cannot fix it	say which field and what is wrong

11.5 Practice exercises

1. Which hook returns the state, the wrapped action and the pending flag in React 19, and from which package is it imported?
2. Which hook reports the pending state of a form, and where must the component that uses it be rendered?
3. Why is server-side validation unavoidable?
4. Which three attributes connect an error message to its input?
5. What does `noValidate` on a form do?

Solutions

1. `useActionState`, imported from `react`.
2. `useFormStatus` from `react-dom`; the component must be a descendant of the `<form>`.
3. Because any client-side check can be skipped by a script, a disabled field, or a hand-made request to the action's endpoint.
4. `aria-invalid` on the input, `aria-describedby` pointing at the message element, and the `id` of that element.
5. It turns off the browser's own validation bubbles, so the messages the server returns are the only ones the user sees.

11.6 Mini test

1. Which hook wraps an action with state handling in React 19? A) `useFormState` from `react-dom` B) `useActionState` from `react` C) `useFormStatus` from `react-dom` D) `useState`
2. What does `useFormStatus` return? A) the form values B) the pending state and metadata of the parent form C) the field errors D) the previous action result
3. Which predicate reports whether a schema parsed a value? A) `parsed.ok` B) `parsed.success` C) `parsed.valid` D) `parsed.done`
4. True or false: disabling the submit button while the action runs prevents a double write.
5. Which attribute marks an input as invalid for a screen reader? A) `aria-live` B) `aria-invalid` C) `aria-hidden` D) `aria-required`
6. What is the first argument of an action used with `useActionState`? A) the event B) the previous state C) the pending flag D) the `FormData`
7. Why should a form that submits to an action be usable without JavaScript? A) it loads faster B) the action still runs, so the mutation is not blocked C) it is easier to test D) it

avoids hydration warnings

Answer key and explanations

1. B - the React 19 hook lives in the `react` package.
2. B - the hook reads the status of the nearest enclosing form.
3. B - `safeParse` returns `success: true` or `false`, and the errors otherwise.
4. True - the pending flag disables the button while the request is in flight.
5. B - `aria-invalid` is the signal the field is wrong, and `aria-describedby` points at the message.
6. B - the signature is `(previousState, formData)`, which is what makes repeated submits meaningful.
7. B - progressive enhancement means the data reaches the server even on a failed bundle.

Chapter 12 - Errors, loading and not-found files

12.1 Summary

- `error.tsx` is a client component that receives `error` and `reset`, and it becomes the error boundary of its own segment.
- An error boundary catches only the failures of the subtree below it, so a thrown error inside a layout is caught by the boundary of the segment above.
- `reset()` renders the segment again, which is what a "Try again" button calls after a transient failure.
- `global-error.tsx` catches the failures of the root layout itself, and because it replaces the whole document it must render its own `<html>` and `<body>`.
- `notFound()` throws a control-flow signal that renders the nearest `not-found.tsx`, which is also what an unknown URL shows.
- A dynamic route usually has two not-found states: the URL does not exist at all, or the record for that id is gone.
- `loading.tsx` and `error.tsx` coexist: the boundary paints the fallback first and replaces it with the error if the render fails.
- In development the overlay shows the full stack; production shows a digest, so the real error should be logged by your own monitoring.
- A server error that reaches the browser without a boundary falls back to the closest one, which is why a root boundary is worth having.

12.2 Concepts

The boundary ladder

output

```
global-error.tsx       catches everything, including the root layout
  app/layout.tsx
    app/error.tsx      catches everything below the root layout
      app/dashboard/error.tsx    catches this segment only
        app/dashboard/page.tsx
```

Each file covers its own segment. A failure deep in the tree is caught as close to it as possible, which is what keeps a broken table from blanking the application.

The props of `error.tsx`

Prop	Type	Use
`error`	an `Error` with a `digest`	log it, show a summary
`reset`	`() => void`	re-render the segment after a failure

The component must be a client component, because it runs in the browser where the failure happened.

`notFound()` and `not-found.tsx`

Situation	What the user sees
A URL with no matching segment	the closest `not-found.tsx`, or the root one
`notFound()` called in a server component	the closest `not-found.tsx` in that segment
A missing record inside a page	whatever you render, or `notFound()` for the file

Calling `notFound()` is a throw under the hood, so code after it does not run.

12.3 Examples

Example 1: a segment error boundary.

TSX

```tsx
// app/dashboard/error.tsx
'use client';

import { useEffect } from 'react';

type Props = { error: Error & { digest?: string }; reset: () => void };

export default function Error({ error, reset }: Props) {
  useEffect(() => {
    console.error(error);
  }, [error]);

  return (
    <div role="alert">
      <h2>Something went wrong</h2>
      <p>Reference {error.digest}</p>
      <button onClick={() => reset()}>Try again</button>
    </div>
  );
}
```

output

```
A failing query inside the dashboard renders this panel in place of the page.
Pressing "Try again" runs the segment again, which is enough for a transient
database failure.
```

Example 2: the global boundary.

TSX

```tsx
// app/global-error.tsx
'use client';

type Props = { error: Error & { digest?: string }; reset: () => void };

export default function GlobalError({ error, reset }: Props) {
  return (
    <html lang="en">
      <body>
        <h1>The application could not start</h1>
        <button onClick={() => reset()}>Reload</button>
      </body>
    </html>
  );
}
```

output

```
This is the last resort. It renders its own document shell, because the layout
that normally provides one is the thing that failed.
```

Example 3: an unknown URL.

TSX

```tsx
// app/not-found.tsx
import Link from 'next/link';

export default function NotFound() {
  return (
    <div>
      <h2>Page not found</h2>
      <p>We could not find the page you asked for.</p>
      <Link href="/">Back to the dashboard</Link>
    </div>
  );
}
```

output

```
Visiting /nope shows this interface with a 404 status code in the response.
```

Example 4: a missing record.

TSX

```tsx
// app/dashboard/invoices/[id]/page.tsx
import { notFound } from 'next/navigation';
import { getInvoiceById } from '@/app/lib/data';

type Props = { params: Promise<{ id: string }> };

export default async function InvoicePage({ params }: Props) {
  const { id } = await params;
  const invoice = await getInvoiceById(id);
  if (!invoice) notFound();
  return <h1>{invoice.name}</h1>;
}
```

output

```
/invoices/99 renders the not-found interface of the closest boundary. The 404
status is sent as well, which is what a search engine expects.
```

Example 5: loading and error together.

output

```
app/dashboard/
  layout.tsx
  loading.tsx        skeleton while the page renders
  error.tsx          replaces the segment if the render throws
  page.tsx           awaits its data
```

output

```
A slow query shows the skeleton. If the same query throws, the skeleton is
replaced by the error panel rather than the table.
```

12.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Writing `error.tsx` as a server component	hooks and the reset button do not work	add `"use client"` to the file
Expecting an error boundary to catch failures in a sibling layout	the boundary covers its own subtree only	place the boundary in the segment that renders the risky data
Swallowing an error with an empty `catch`	the user sees a stale screen and you lose the report	log it and return a visible state
Showing the raw error message to users	it leaks internals and confuses people	show a digest or a friendly sentence
Returning 200 for a missing record	search engines index nothing	call `notFound()` so the status is 404
Forgetting `reset` in the UI	the user must reload the whole tab	offer the retry button

12.5 Practice exercises

1. Which two props does `error.tsx` receive, and what does each do?
2. Why must `error.tsx` be a client component?
3. What does `global-error.tsx` have to render that no other special file has to?
4. Which function produces the 404 interface for a missing invoice?
5. A query fails once because the database restarts. Which two things should the boundary file do?

Solutions

1. `error`, the error object to log and summarise, and `reset`, the function that re-renders the segment.
2. Because it renders in the browser where the failure happened, and because `reset` and `useEffect` only exist on the client.
3. Its own `<html>` and `<body>` elements, since it replaces the document when the root layout itself fails.
4. `notFound()` from `next/navigation`, which renders the closest `not-found.tsx` and sets a 404 status.
5. Log the error, and offer a "Try again" button that calls `reset()`.

12.6 Mini test

1. Which file is the error boundary of a route segment? A) `not-found.tsx` B) `error.tsx` C) `loading.tsx` D) `template.tsx`
2. Which two props does `error.tsx` receive? A) `error` and `reset` B) `children` and `params` C) `state` and `dispatch` D) `error` and `children`
3. True or false: an error boundary catches failures that happen in a layout above it.
4. Which function renders the not-found interface for a missing record? A) `redirect` B) `notFound` C) `revalidatePath` D) `error`
5. Why does `global-error.tsx` render its own `<html>` element? A) it is a React requirement B) it replaces the document when the root layout fails C) it improves SEO D) it is required by TypeScript
6. What does the `reset` function do? A) clears the router cache B) renders the segment again C) reloads the browser tab D) runs the action again
7. What does the browser receive when `notFound()` runs? A) a 500 response B) a 404 response with the not-found interface C) a redirect to `/` D) an empty page

Answer key and explanations

1. B - `error.tsx` is the file convention for the segment's error boundary.
2. A - the boundary receives the error and the retry function.
3. False - a boundary only covers its own subtree.

4. B - `notFound()` throws the signal that renders the closest not-found file.
5. B - the root layout is gone, so the boundary has to provide the document shell.
6. B - `reset` re-renders the segment, which recovers from transient failures.
7. B - the status code matters to search engines and to monitoring.

Chapter 13 - Authentication and protected routes

13.1 Summary

- Authentication answers who the user is; authorization answers what that user may do, and both must be decided on the server.
- A session is usually a cookie the browser sends with every request, and the server reads it with `await cookies()`.
- Reading the session in a page or layout is enough to gate a whole area, because the children never render when the parent redirects.
- A Server Action must re-check the session and the ownership of the row, because an action is a public endpoint.
- Middleware runs before a route is rendered, so it is the cheapest place to protect a group of paths with one rule.
- The `matcher` option decides which requests enter the middleware, and static assets are normally excluded from it.
- Libraries such as Auth.js wrap providers, callbacks and session handling, and export a helper that can be called on the server and from middleware.
- A route handler can act as a proxy, adding the session cookie before it forwards the request to a protected backend.
- Hiding a button is a hint for the interface, never a security rule.

13.2 Concepts

Where a check can live

Place	Cost	Good for
Middleware	runs before rendering	protecting whole path groups
Page or layout	runs once per request	gating a section with a redirect
Server Action	runs per call	every mutation, without exception
Route handler	runs per request	API endpoints and proxies
Client component	free, and untrusted	hiding what the user cannot do

Reading a session on the server

TypeScript

```ts
import { cookies } from 'next/headers';

export async function getSession() {
  const jar = await cookies();
  const token = jar.get('session')?.value;
  if (!token) return null;
  return verifySessionToken(token);
}
```

In version 15 `cookies()` is asynchronous, so the call is awaited before the value is read. Anything that reads the session makes its route dynamic, because the answer differs per request.

The middleware layer

Concept	Notes
Location	`middleware.ts` at the project root or beside `src/`
Export	a default function or a wrapped one from the library
`config.matcher`	the list of paths the function runs for
Order	the request, then the middleware, then the route
Response	`NextResponse.next()`, `redirect`, or `rewrite`

Forbidden and unauthorized responses

Recent minor versions add file conventions that answer 403 and 401 instead of a redirect. They arrived as experimental features, so the exact availability depends on the release you installed.

13.3 Examples

Example 1: a session helper.

TypeScript

```ts
// app/lib/session.ts
import { cookies } from 'next/headers';

export type Session = { userId: string; email: string };

export async function getSession(): Promise<Session | null> {
  const jar = await cookies();
  const raw = jar.get('session')?.value;
  if (!raw) return null;
  try {
    return JSON.parse(Buffer.from(raw, 'base64').toString('utf8')) as Session;
  } catch {
    return null;
  }
}
```

output

```
An anonymous visitor gets null, and a signed-in visitor gets the session
object. Real projects verify a signature on the value instead of trusting the
decoded body; a library does that for you.
```

Example 2: gating a section with a redirect.

TSX

```tsx
// app/dashboard/layout.tsx
import { redirect } from 'next/navigation';
import { getSession } from '@/app/lib/session';

export default async function DashboardLayout({ children }: {
  children: React.ReactNode;
}) {
  const session = await getSession();
  if (!session) redirect('/login');
  return <section>{children}</section>;
}
```

output

```
Every URL under /dashboard is protected by one file. An anonymous request for
/dashboard/invoices is redirected to /login, and the page inside is never
rendered.
```

Example 3: middleware for a group of paths.

TypeScript

```ts
// middleware.ts
import { NextResponse } from 'next/server';

export function middleware(request: Request) {
  const hasSession = request.headers.get('cookie')?.includes('session=');
  if (!hasSession) {
    return NextResponse.redirect(new URL('/login', request.url));
  }
  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*', '/settings/:path*'],
};
```

output

```
A request for /dashboard/invoices without a session cookie is redirected before
the page runs. The matcher keeps the function away from the assets and the API
routes you want to leave open.
```

Example 4: authorising a mutation.

TypeScript

```ts
// app/lib/actions.ts
'use server';

import { revalidatePath } from 'next/cache';
import { getSession } from '@/app/lib/session';

export async function deleteInvoice(formData: FormData) {
  const session = await getSession();
  if (!session) return { errors: { form: ['Please log in first'] } };
  const id = String(formData.get('id') ?? '');
  await sql`DELETE FROM invoices WHERE id = ${id} AND user_id = ${session.userId}`;
  revalidatePath('/dashboard/invoices');
  return { errors: {} };
}
```

output

```
The ownership condition lives in the query, so a signed-in user cannot delete
another user's row by changing the id in the markup.
```

Example 5: protecting a route handler.

TypeScript

```ts
// app/api/invoices/route.ts
import { NextResponse } from 'next/server';
import { getSession } from '@/app/lib/session';

export async function GET() {
  const session = await getSession();
  if (!session) {
    return NextResponse.json({ message: 'Unauthorised' }, { status: 401 });
  }
  return NextResponse.json(await getInvoicesForUser(session.userId));
}
```

shell

```sh
curl -i http://localhost:3000/api/invoices
```

output

```
HTTP/1.1 401 Unauthorized
content-type: application/json

{"message":"Unauthorised"}
```

Example 6: what a library contributes.

Concern	What it handles
Providers	OAuth and credentials sign-in flows
Callbacks	sign-in, redirect, session and JWT hooks
Session	reading and writing the cookie for you
Middleware	a wrapper you export instead of writing the check
Versioning	Pages Router and App Router use different major versions

13.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Hiding a delete button instead of checking	any user can call the endpoint	check the session in the action
Checking only in middleware	a route can be reached another way	check again in the page or the action
Trusting an id from the client	users can edit another row's data	filter by the session's user id in the query
Reading `cookies()` synchronously in version 15	the call fails or returns a promise	`await cookies()`
Putting the whole app behind middleware	static assets and APIs get slow	narrow the matcher
Storing the password in the session	a leaked cookie is a leaked credential	store an opaque session token

13.5 Practice exercises

1. Which two questions do authentication and authorization answer, and where must both be decided?
2. How do you protect every page under `/dashboard` with a single file?
3. Why must a Server Action re-check the session?
4. What does `config.matcher` control in `middleware.ts`?
5. In version 15, how is `cookies()` read?

Solutions

1. Authentication answers who the user is, and authorization answers what that user may do; both are decided on the server.
2. Add `app/dashboard/layout.tsx` that reads the session and calls `redirect('/login')` when it is missing.
3. Because an action is a public endpoint that can be called directly, so the check cannot live only in the interface.
4. The list of paths the middleware runs for, which keeps it away from static assets and open APIs.
5. It is an asynchronous function, so you write `const jar = await cookies();`.

13.6 Mini test

1. Which module provides the `cookies` function? A) `next/cache` B) `next/headers` C) `next/navigation` D) `next/server`

2. What is the cheapest place to protect a whole group of paths? A) `loading.tsx` B) middleware with a matcher C) a `<Link>` D) an error boundary
3. True or false: hiding a delete button in the interface protects the data.
4. Why does reading a session make a route dynamic? A) it uses a database B) the answer differs per request C) it is a client component D) it disables caching of `fetch`
5. Which HTTP status signals an unauthenticated request to an endpoint? A) 302 B) 401 C) 403 D) 404
6. What should the `WHERE` clause of a delete contain when the row has an owner? A) the id only B) the id and the session's user id C) the user id only D) nothing, the action is protected
7. In Next.js 15, which shape does the cookies helper have? A) a synchronous object B) an asynchronous function C) a hook D) a class

Answer key and explanations

1. B - `next/headers` is where request data is read on the server.
2. B - one middleware covers every matched path before any rendering happens.
3. False - the markup is only a hint; the endpoint stays callable.
4. B - per-request data cannot be shared between visitors, so the route is rendered per request.
5. B - 401 means no valid authentication was supplied.
6. B - the ownership condition belongs in the query itself.
7. B - version 15 made the request helpers asynchronous.

Chapter 14 - Metadata, SEO and deployment

14.1 Summary

- The metadata API sets titles and descriptions through two exports: a static `metadata` object and an asynchronous `generateMetadata` function.
- A child route's metadata merges with its parent's, and a parent can declare a title template so children only supply the page part.
- In version 15 `generateMetadata` receives `params` and `searchParams` as promises, so both are awaited before the values are used.
- Icons, open graph images, sitemaps and robots files come from file conventions or from exported generator functions.
- Viewport settings live in their own `viewport` export rather than inside `metadata`.
- `metadataBase` gives the framework a base URL to resolve relative links for social previews.
- Static routes emit finished HTML while building, which is what a crawler reads; dynamic routes emit HTML on request.

- `next build` writes everything into `.next`, and `next start` serves that folder with Node.
- `output: 'standalone'` produces a self-contained server that can be copied into a container image or a bare virtual machine.
- Environment variables are server-only unless they are prefixed with `NEXT_PUBLIC_`, which inlines them into the browser bundle.

14.2 Concepts

Where the checks happen: the two exports

Export	Shape	Use
`metadata`	an object	values that never change for the route
`generateMetadata`	an async function	values that depend on params or on data

Inheritance and templates

output

```
app/layout.tsx            title: { default: 'Acme', template: '%s | Acme' }
  app/blog/layout.tsx     title: 'Blog'
    app/blog/[slug]/page.tsx   title: 'Hello world'

result:                              Hello world | Acme
```

The order is child over parent. A `title` that is a plain string replaces the parent's value; a template keeps it as a suffix.

File conventions

File	Result
`app/favicon.ico`	the site icon
`app/icon.png`, `app/apple-icon.png`	platform icons
`app/opengraph-image.tsx`	a generated social image
`app/sitemap.ts`	`/sitemap.xml`
`app/robots.ts`	`/robots.txt`

Building and running

Command	Effect
`npm run build`	production build into `.next`
`npm run start`	serve the build on port 3000
`output: 'standalone'`	emit a minimal folder with its own server
`NEXT_PUBLIC_API_URL=... npm run build`	inline a value into the browser bundle

The minimum Node version is documented in the framework's requirements page; Next.js 15 expects a current long-term support release.

14.3 Examples

Example 1: metadata in the root layout.

TypeScript

```
// app/layout.tsx
import type { Metadata } from 'next';

export const metadata: Metadata = {
  metadataBase: new URL('https://acme.example'),
  title: { default: 'Acme invoicing', template: '%s | Acme invoicing' },
  description: 'Create invoices and track payments',
  openGraph: { title: 'Acme invoicing', type: 'website' },
};
```

output

```
Every page inherits the description, and a page that declares its own title
renders it as "Invoices | Acme invoicing" in the tab.
```

Example 2: metadata for a dynamic route.

TSX

```
// app/invoices/[id]/page.tsx
import type { Metadata } from 'next';
import { getInvoiceById } from '@/app/lib/data';

type Props = { params: Promise<{ id: string }> };

export async function generateMetadata({ params }: Props): Promise<Metadata> {
  const { id } = await params;
  const invoice = await getInvoiceById(id);
  return {
    title: invoice ? invoice.name : 'Invoice not found',
    description: 'Details of one invoice',
  };
}

export default async function InvoicePage({ params }: Props) {
  const { id } = await params;
  const invoice = await getInvoiceById(id);
  return <h1>{invoice?.name}</h1>;
}
```

output

```
The tab of /invoices/2 reads "Globex | Acme invoicing". Nothing is written to
the page's markup: the metadata ends up in the <head> element.
```

Example 3: a sitemap and a robots file.

TypeScript

```ts
// app/sitemap.ts
import type { MetadataRoute } from 'next';

export default function sitemap(): MetadataRoute.Sitemap {
  return [
    { url: 'https://acme.example', lastModified: new Date('2026-01-01') },
    { url: 'https://acme.example/invoices', lastModified: new Date('2026-01-01') },
  ];
}
```

TypeScript

```ts
// app/robots.ts
import type { MetadataRoute } from 'next';

export default function robots(): MetadataRoute.Robots {
  return {
    rules: { userAgent: '*', allow: '/' },
    sitemap: 'https://acme.example/sitemap.xml',
  };
}
```

shell

```
curl -s http://localhost:3000/sitemap.xml
```

output

```
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url><loc>https://acme.example</loc></url>
  <url><loc>https://acme.example/invoices</loc></url>
</urlset>
```

Example 4: a standalone build for a container.

TypeScript

```ts
// next.config.ts
import type { NextConfig } from 'next';

const nextConfig: NextConfig = { output: 'standalone' };

export default nextConfig;
```

Dockerfile

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/public ./public
COPY --from=build /app/.next/standalone ./
COPY --from=build /app/.next/static ./.next/static
EXPOSE 3000
CMD ["node", "server.js"]
```

output

```
The build produces .next/standalone with a server.js entry point and only the
files that were traced as used. The second stage copies that folder and needs
no install step.
```

Example 5: environment variables.

shell

```shell
NEXT_PUBLIC_API_URL=https://api.acme.example npm run build
```

TypeScript

```typescript
// available in the browser
console.log(process.env.NEXT_PUBLIC_API_URL);

// available only in server components, actions and route handlers
console.log(process.env.DATABASE_URL);
```

output

```
The public value is inlined into the bundle, so it is visible to anyone who
opens the developer tools. The database variable stays on the server and is
undefined in the browser.
```

Example 6: the build and the start.

shell

```shell
npm run build
npm run start
```

output

```
 ✓ Compiled successfully
 ✓ Collecting page data
 ✓ Generating static pages (7/7)

Route (app)                    Size  First Load JS
┌ ○ /                         132 B         102 kB
├ ● /invoices                 132 B         102 kB
│   ISR: revalidate every 3600 seconds
└ λ /dashboard                132 B         102 kB

   ▲ Next.js 15.x
   - Local:        http://localhost:3000
```

14.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
A missing `metadataBase`	social previews use relative paths	set it once in the root layout
Reading `params.id` in `generateMetadata` on version 15	the title breaks	`const { id } = await params`

Mistake	Why it hurts	Best practice
Overriding `metadata` instead of merging	a parent's description disappears	export what changes
Putting the viewport in `metadata`	the setting is ignored	use the `viewport` export
Building a private value into `NEXT_PUBLIC_`	it leaks into the bundle	keep it server-only
Running `next start` against a stale build	the served code is not the built code	build, then start
Assuming a single host is required	the output is portable Node	the same build runs on a VM, a container or a platform

14.5 Practice exercises

1. Which two exports set metadata, and when do you use each?
2. How does a title template travel from a parent layout to a child page?
3. In version 15, how do you read the `id` inside `generateMetadata`?
4. Which configuration option produces a folder with its own server for a container?
5. What is the difference between `DATABASE_URL` and `NEXT_PUBLIC_API_URL`?

Solutions

1. The static `metadata` object for fixed values, and `generateMetadata` when the values depend on params or on data.
2. The parent declares `title: { default, template }`, and the child supplies a plain string that is inserted into the template.
3. `const { id } = await params;`, because the props are promises in version 15.
4. `output: 'standalone'` in `next.config.ts`.
5. The first is read only on the server, while the second is inlined into the browser bundle and therefore public.

14.6 Mini test

1. Which export provides metadata for a dynamic route? A) `metadata` B) `generateMetadata` C) `useMetadata` D) `Head`
2. What does `title.template` do? A) duplicates the title B) wraps child titles with a pattern C) removes the parent title D) sets the favicon
3. In Next.js 15, what are the props of `generateMetadata`? A) promises B) plain objects C) functions D) strings
4. Which export declares viewport settings? A) `metadata` B) `viewport` C) `head` D) `layout`
5. Which build output can be copied into a container without an install step? A) `.next/cache` B) `.next/standalone` C) `.next/server/pages` D) `public/`

6. Which prefix makes an environment variable visible in the browser? A) `PUBLIC_` B) `NEXT_PUBLIC_` C) `CLIENT_` D) `EXPOSE_`
7. What does `npm run start` do? A) runs the dev server B) serves the production build with Node C) rebuilds the project D) removes `.next`
8. Why should `metadataBase` be set? A) it speeds up the build B) it gives relative links a base for social previews C) it generates the sitemap D) it disables analytics

Answer key and explanations

1. B - the async function is the one that may read `params`.
2. B - the template in a parent wraps the title of its descendants.
3. A - version 15 made the metadata props promises as well.
4. B - viewport settings have their own export.
5. B - the standalone output bundles a small server with the traced files.
6. B - only prefixed variables are inlined into the bundle, and they become public.
7. B - `start` serves what `build` produced.
8. B - without it, relative URLs in open graph tags cannot be resolved by a crawler.

Cheat sheet

15.1 Commands

Command	Effect
`npx create-next-app@latest app --ts --tailwind --eslint --app`	scaffold a project
`npm run dev`	dev server on port 3000
`npm run build`	production build into `.next`
`npm run start`	serve the production build
`npm run lint`	run ESLint

15.2 Files and conventions

File	Meaning
`app/layout.tsx`	shell of the segment, renders `children`
`app/page.tsx`	the route itself
`app/loading.tsx`	Suspense fallback for the segment
`app/error.tsx`	client error boundary, receives `error` and `reset`

File	Meaning
`app/global-error.tsx`	root boundary, must render `<html>` and `<body>`
`app/not-found.tsx`	interface for `notFound()` and unknown URLs
`app/route.ts`	HTTP endpoint exporting `GET`, `POST`, ...
`app/sitemap.ts`	generates `/sitemap.xml`
`app/robots.ts`	generates `/robots.txt`
`[id]`, `[...slug]`, `[[...slug]]`	dynamic, catch-all, optional catch-all
`(group)`	route group, adds no URL segment
`_private`	excluded from routing
`middleware.ts`	runs before the route, with `config.matcher`

15.3 Components and navigation

Task	Code
Navigate	`<Link href="/invoices">Invoices</Link>`
Replace history	`<Link replace href="/login">`
Keep scroll position	`<Link scroll={false} href="/x">`
No prefetch	`<Link prefetch={false} href="/x">`
Current path	`const pathname = usePathname()`
Query string on the client	`useSearchParams().get('query')`
Navigate from code	`router.push('/invoices')`
Re-fetch the route	`router.refresh()`
Redirect from the server	`redirect('/login')`

15.4 Rendering and caching

Need	Write
Force static	`export const dynamic = 'force-static'`
Force dynamic	`export const dynamic = 'force-dynamic'`
Prerender known params	`export async function generateStaticParams()`
Time-based cache	`export const revalidate = 3600`
Reject unknown params	`export const dynamicParams = false`
Cache one request	`fetch(url, { next: { revalidate: 60 } })`

Need	Write
Tag one request	`fetch(url, { next: { tags: ['invoices'] } })`
Invalidate a route	`revalidatePath('/invoices')`
Invalidate a tag	`revalidateTag('invoices')`
Dynamic APIs	`await cookies()`, `await headers()`, `await searchParams`, `connection()`
Stream a slow region	`<Suspense fallback={<Skeleton />}>`
Parallel fetching	`Promise.all([a(), b()])`
Deduplicate per request	`cache(async () => ...)` from `react`

15.5 Mutations and forms

Task	Code
Declare an action	`'use server';` above the function or file
Read a field	`formData.get('name')`
Form action	`<form action={createInvoice}>`
State + action + pending	`const [state, action, pending] = useActionState(fn, initial)`
Form pending state	`const { pending } = useFormStatus()`
Action signature	`(previousState, formData) => nextState`
Validate	`Schema.safeParse({...})` then `error.flatten().fieldErrors`
Disable the submit button	`<button disabled={pending}>`
Report an error	`aria-invalid`, `aria-describedby`, `id`, `aria-live`
Check permissions	`const session = await getSession()` inside the action
Protect a section	a layout that redirects when the session is missing

15.6 Metadata

Need	Write
Static metadata	`export const metadata: Metadata = { ... }`
Dynamic metadata	`export async function generateMetadata({ params })`
Title template	`` `title: { default: 'Acme', template: '%s Acme' }` ``
Base URL	`metadataBase: new URL('https://acme.example')`
Viewport	`export const viewport: Viewport = { ... }`

Need	Write
Icons	`app/favicon.ico`, `app/icon.png`, `app/apple-icon.png`
Social image	`app/opengraph-image.tsx`

15.7 Version 15 at a glance

Area	Version 15
`params` and `searchParams` in pages, layouts, handlers	promises, `await` them
`cookies()`, `headers()`, `draftMode()`	asynchronous
`fetch` and GET route handlers	not cached by default
Form state hook	`useActionState` from `react`
Client router cache	no stale time by default
Low-level escape hatch	`connection()` from `next/server`

15.8 Styling, fonts and images

Task	Write
Google font	`const inter = Inter({ subsets: ['latin'] })` then `className={inter.className}`
Local font	`localFont({ src: './x.woff2', variable: '--font-brand' })`
Sized image	`<Image src={hero} alt="Revenue chart" priority />`
Filling image	`<Image src="https://..." alt="Team" fill sizes="(max-width: 768px) 100vw, 50vw" />`
Remote images allowed	`images.remotePatterns` in `next.config.ts`
Scoped styles	`import styles from './card.module.css'`

Final test

1. Which folder holds the files of the App Router? A) `pages/` B) `app/` C) `src/router/` D) `components/`
2. Which first line marks a file as a Client Component? A) `"use server"` B) `"use client"` C) `"use browser"` D) `"use strict"`
3. True or false: in Next.js 15 the `params` prop of a page component must be awaited.
4. Which file wraps a whole route segment in a Suspense boundary? A) `error.tsx` B) `loading.tsx` C) `template.tsx` D) `suspense.tsx`

5. What does `revalidatePath('/invoices')` do after a mutation? A) deletes the route B) discards the cached output of that route C) rebuilds the whole site D) redirects to `/invoices`
6. Which of these may a `route.ts` file export? A) only `GET` B) `GET`, `POST`, `PATCH`, `DELETE` and more C) a React component D) `metadata`
7. What does `<Link>` do that a plain anchor does not? A) renders a `button` B) navigates without a full document reload C) adds a `target` attribute D) disables the back button
8. Which hook reports the current URL path to a client component? A) `useRouter` B) `searchParams` C) `usePathname` D) `headers`
9. Where must the permission check of a Server Action be performed? A) in the client component B) inside the action on the server C) in the CSS D) in middleware only
10. What is the signature of an action used with `useActionState`? A) `(formData) => void` B) `(previousState, formData) => nextState` C) `(event) => Promise` D) `(state, dispatch) => void`
11. What does `useFormStatus` report? A) the values of the form fields B) the pending state of the nearest parent form C) the field errors D) the last submitted `FormData`
12. Which of these makes a route dynamic? A) `generateStaticParams` B) `await cookies()` C) `export const revalidate` D) `metadata`
13. What is the difference between `next/font/google` and `next/font/local`?
14. Which props does a remote `next/image` need in order to size itself? A) `width` and `height` B) `fill` with a sized parent C) `priority` only D) `sizes` only
15. What does `output: 'standalone'` add to the build? A) a zip file B) a minimal folder with its own server entry point C) a container image D) a static export of every route
16. Which prefix exposes an environment variable to the browser? A) `PUBLIC_` B) `NEXT_PUBLIC_` C) `CLIENT_` D) `INLINE_`
17. What does `generateStaticParams` return? A) a list of metadata objects B) the list of known param values for a dynamic segment C) the HTML of the page D) the sitemap
18. What does the marker next to a dynamic route in the build output look like? A) a filled circle B) a lambda C) a triangle D) a square
19. Which file decides which requests enter the middleware? A) `next.config.ts` B) the `config.matcher` export of `middleware.ts` C) `app/middleware.ts` D) `tsconfig.json`
20. What does the user see when a page calls `notFound()`? A) the error boundary B) the closest `not-found.tsx`, with a 404 status C) a blank page with a 200 status D) the loading skeleton forever
21. What does `Promise.all([getA(), getB()])` fix in a server component?
22. True or false: in Next.js 15 a plain `fetch(url)` is cached for 30 seconds by default.
23. Which file catches the failures of the root layout itself? A) `app/error.tsx` B) `app/global-error.tsx` C) `app/not-found.tsx` D) `app/loading.tsx`
24. What are the two props of `error.tsx`, and what does the second one do?
25. A route under `/dashboard` must only be visible to signed-in users. Name two places where the check belongs.

16.1 Final test answers

1. B - the App Router reads every route from the `app/` folder.
2. B - the `"use client"` directive opens the client bundle boundary.
3. True - version 15 made `params` a promise, so `const { id } = await params` is the correct form.
4. B - `loading.tsx` is the file convention for the segment's Suspense fallback.
5. B - the cached route output is discarded, so the next render fetches fresh data.
6. B - one function per HTTP verb may be exported.
7. B - the click is intercepted and only the changed segments are rendered again.
8. C - `usePathname` returns the path of the current URL.
9. B - an action is a public endpoint, so the server must re-check the session and the ownership.
10. B - the helper passes the previous state as the first argument, which is how repeated submits report their outcome.
11. B - the hook reports the state of the nearest enclosing `<form>`.
12. B - cookies differ per request, so the route cannot be shared between visitors.
13. `next/font/google` downloads a font from the Google library while building and self-hosts it; `next/font/local` serves a file that is already in the repository.
14. A or B - either intrinsic dimensions, or `fill` inside a parent that has a position and a size.
15. B - the standalone output contains a `server.js` entry point and the traced files only.
16. B - only prefixed variables are inlined into the bundle, and therefore public.
17. B - for example `[{ id: '1' }, { id: '2' }]` for a segment named `[id]`.
18. B - a lambda marks a route that is rendered per request.
19. B - the matcher lists the paths the middleware runs for.
20. B - the signal is caught by the nearest `not-found.tsx` and the status code is 404.
21. A waterfall: the two requests used to run one after the other, and the page waited for both latencies.
22. False - version 15 removed the default `fetch` cache, so the request is not cached unless you ask.
23. B - `global-error.tsx` replaces the document and renders its own `<html>` and `<body>`.
24. `error`, the object to log and summarise, and `reset`, which renders the segment again after a transient failure.
25. In the layout or page that renders the section, with a redirect when the session is missing, and inside every Server Action that mutates data. Middleware is a reasonable third place for the path group.

Sources

Consulted while writing this volume:

- Learn Next.js, the course index: https://nextjs.org/learn

- React Foundations track: https://nextjs.org/learn/react-foundations
- Pages Router track, the legacy router: https://nextjs.org/learn/pages-router
- Dashboard App track, the main course: https://nextjs.org/learn/dashboard-app
- Getting started chapter: https://nextjs.org/learn/dashboard-app/getting-started
- CSS styling chapter: https://nextjs.org/learn/dashboard-app/css-styling
- Optimizing fonts and images chapter: https://nextjs.org/learn/dashboard-app/optimizing-fonts-images
- Creating layouts and pages chapter: https://nextjs.org/learn/dashboard-app/creating-layouts-and-pages
- Navigating between pages chapter: https://nextjs.org/learn/dashboard-app/navigating-between-pages
- Setting up your database chapter: https://nextjs.org/learn/dashboard-app/setting-up-your-database
- Fetching data chapter: https://nextjs.org/learn/dashboard-app/fetching-data
- Static and dynamic rendering chapter: https://nextjs.org/learn/dashboard-app/static-and-dynamic-rendering
- Streaming chapter: https://nextjs.org/learn/dashboard-app/streaming
- Adding search and pagination chapter: https://nextjs.org/learn/dashboard-app/adding-search-and-pagination
- Mutating data chapter: https://nextjs.org/learn/dashboard-app/mutating-data
- Error handling chapter: https://nextjs.org/learn/dashboard-app/error-handling
- Improving accessibility chapter: https://nextjs.org/learn/dashboard-app/improving-accessibility
- Adding authentication chapter: https://nextjs.org/learn/dashboard-app/adding-authentication
- Adding metadata chapter: https://nextjs.org/learn/dashboard-app/adding-metadata
- Next steps chapter: https://nextjs.org/learn/dashboard-app/next-steps
- Documentation, the App Router overview: https://nextjs.org/docs/app
- Project structure reference: https://nextjs.org/docs/app/getting-started/project-structure
- The `Link` component reference: https://nextjs.org/docs/app/api-reference/components/link
- The `Image` component reference: https://nextjs.org/docs/app/api-reference/components/image
- The font helpers reference: https://nextjs.org/docs/app/api-reference/functions/next-font
- Page and layout file conventions: https://nextjs.org/docs/app/api-reference/file-conventions/page
- Route handler file convention: https://nextjs.org/docs/app/api-reference/file-conventions/route
- `generateMetadata` reference: https://nextjs.org/docs/app/api-reference/functions/generate-metadata

- `generateStaticParams` reference: https://nextjs.org/docs/app/api-reference/functions/generate-static-params
- Server Actions and mutations guide: https://nextjs.org/docs/app/building-your-application/data-fetching/server-actions-and-mutations
- Route handlers guide: https://nextjs.org/docs/app/building-your-application/routing/route-handlers
- Authentication guide: https://nextjs.org/docs/app/building-your-application/authentication
- Deployment guide: https://nextjs.org/docs/app/building-your-application/deploying
- The version 15 upgrade guide, for the breaking changes: https://nextjs.org/docs/app/guides/upgrading/version-15
- `next.config.js` options reference: https://nextjs.org/docs/app/api-reference/config/next-config-js
- Tailwind CSS documentation, for the styling chapter: https://tailwindcss.com/docs
- Zod documentation, for the validation chapter: https://zod.dev/